

2026 전국기능경기대회 클라우드컴퓨팅 제2과제

AWS 콘솔 상세 해설서

초보자를 위한 단계별 구축과 검증

클릭 경로와 입력값부터 정상 결과와 오류 해결까지, 다섯 과제를 한 단계씩 따라 합니다. 복사용 코드와 설정 파일을 함께 제공합니다.

Workflow · Real time data analytics · MSK
CDN Service Setup · Legacy System Operation

총 30점 | 경기시간 4시간 | 학습용 상세판

원문 충돌 조건은 별도로 표시했습니다. 실제 AWS 계정 배포와 채점 결과를 보장하는 문서는 아닙니다.

목차

시작 안내와 원문 확인	3
1 공통 콘솔 조작과 네트워크	7
2 Workflow 상세 구축	12
3 Real time data analytics 상세 구축	17
4 MSK 상세 구축	22
5 CDN Service Setup 상세 구축	29
6 Legacy System Operation 상세 구축	34
7 제출 전 채점 항목별 확인	44
8 초보자를 위한 오류 해결 사전	47
9 참고 자료와 검증 범위	49

시작 안내와 원문 확인

AWS 콘솔로 따라 하는 초보자용 실습 안내서

이 해설서는 Small Challenge의 다섯 과제를 직접 구성하고, 정상 동작을 확인하는 방법을 설명합니다. 먼저 공통 작업을 익히고 Workflow부터 차례로 연습하세요. 각 단계의 완료 확인을 통과한 다음 단계로 넘어가면 오류가 난 위치를 쉽게 찾을 수 있습니다.

리소스는 AWS 웹 콘솔에서 만듭니다. 운영체제에 파일을 넣고 앱을 실행하는 작업은 콘솔 안의 Session Manager, 파일 압축과 이미지 빌드는 CloudShell 또는 콘솔로 만든 작업용 EC2에서 합니다. IaC나 전체 자동 배포를 전제로 하지 않습니다.

읽는 순서와 제공 파일

순서	내용	목표
1	공통 준비	리전, IAM, 보안 그룹, VPC, 파일 전달 이해
2	Workflow	CSV 업로드가 자동으로 처리되게 구성
3	Real time data analytics	EC2 주문 데이터를 Kinesis와 SQL로 분석
4	MSK	센서 데이터 소비, 이상 탐지, 저장과 알림
5	CDN Service Setup	정적 파일과 현재 시간 API를 단일 도메인으로 제공
6	Legacy System Operation	지급 앱을 ECS와 Lambda에서 실행하고 성능 개선
7	제출 전 점검	채점 항목별 확인과 시험 데이터 정리

동봉한 실습파일 폴더에는 복사해서 사용할 Python, JSON, SQL, systemd 설정, Dockerfile이 있습니다. 파일명으로 검색하면 사용하는 단계가 나옵니다. 원본 지급파일은 변경하지 않습니다. 코드 파일을 Word나 PDF에서 복사하면 들여쓰기가 달라질 수 있으므로, 실습파일을 메모장이나 VS Code로 열어 복사하세요.

이 문서는 사진 10쪽과 채점표 2장, 같은 대회 자료 폴더의 day2_release_files 지급파일을 대조했습니다. 지급파일의 앱 명세와 바이너리 단서를 반영했으며, 작성한 예제는 로컬 문법·로직 점검을 거쳤습니다. 실제 AWS 계정에 배포하거나 30점 채점을 실행한 결과는 아닙니다. 참고 문서 확인일은 2026년 9월 29일입니다.

먼저 알아야 할 용어

용어	쉬운 설명	이 문제에서 쓰는 곳
리전	AWS 서버가 위치한 지역	과제마다 반드시 다르게 선택
AZ	한 리전 안의 분리된 데이터센터 구역	ALB와 MSK를 서로 다른 두 구역에 배치

용어	쉬운 설명	이 문제에서 쓰는 곳
VPC	내 서비스 전용 사설 네트워크	EC2, MSK, Aurora, ECS
서브넷	VPC를 나눈 주소 구역	공개 입구와 내부 서버 구역 분리
IGW	VPC에서 인터넷으로 나가는 출입구	공개 서브넷의 기본 경로
NAT Gateway	내부 서버가 외부로 요청하게 하는 장치	패키지 설치, AWS API 호출
보안 그룹	연결을 허용하는 방화벽	ALB만 EC2 앱 포트에 접근
IAM 역할	서비스가 AWS API를 호출할 때 쓰는 권한	EC2가 Kinesis에 기록
ARN	리소스의 전 세계 고유 주소	IAM 정책과 서비스 연결
엔드포인트	접속할 DNS 이름	ALB, DB, Kafka, 캐시
타겟 그룹	ALB가 요청을 보낼 대상 묶음	EC2, ECS, Lambda
환경변수	앱에 전달하는 설정값	버킷 이름, 테이블 이름

보안 그룹과 IAM은 역할이 다릅니다. 보안 그룹에서 통신이 허용되어도 IAM 권한이 없으면 API 요청은 AccessDenied로 실패합니다. IAM이 올바르더라도 네트워크 경로가 없으면 연결 시간이 초과됩니다.

과제별 리전과 배점

과제	콘솔에서 선택할 리전	배점
Workflow	싱가포르 ap-southeast-1	7
Real time data analytics	서울 ap-northeast-2	7
MSK	도쿄 ap-northeast-1	7
CDN Service Setup	버지니아 북부 us-east-1	3
Legacy System Operation	프랑크푸르트 eu-central-1	6

IAM과 CloudFront는 전역 서비스입니다. CloudFront 화면에서 리전 선택이 없는 것은 정상입니다. 연결하는 S3·KMS·API Gateway·Lambda는 CDN 과제에서 us-east-1로 만듭니다. 리소스가 보이지 않으면 삭제되었다고 생각하기 전에 오른쪽 위 리전을 확인하세요.

원문에서 혼동하기 쉬운 조건

이름 철자. Workflow·Analytics·MSK는 wsc2026이고 CDN의 배포 Comment, 버킷, API, Lambda 이름은 wsk2026입니다. 한 글자 차이를 임의로 통일하지 마세요. Legacy는 shgold 접두사를 씁니다.

삭제된 과제. 표지의 Cloud Event Handling 삭제 안내는 그 이름의 별도 과제를 가리킵니다. 현재 본문과 채점표에 남은 MSK 7점까지 삭제한다는 의미로 해석하지 않습니다.

CloudFront OAI와 KMS. 원문은 OAI와 고객 관리 KMS 키를 동시에 요구하지만 OAI는 SSE-KMS 객체 읽기를 지원하지 않습니다. 일반적인 직접 S3 오리진 방식으로 두 조건을 동시에 만족할 수 없습니다. 기술적으로 동작하는 OAC와 SSE-KMS 대안을 이 책에서 설명합니다. OAI 채점 항목의 대체 인정 여부는 대회 정정 기준이 필요합니다. 단순히 OAI를 추가로 만들어 두는 것으로 해결되지 않습니다. [공식 근거 S1]

Flink 버전. 원문은 Studio Notebook, Flink 1.19, ZEPPELIN-FLINK-3_0을 함께 적었습니다. 공식 Flink 1.19 문서는 Studio의 1.19 지원 제한과 1.15 사용을 안내합니다. 콘솔에서 가능한 Studio 조합을 확인하고 정정 기준을 적용해야 합니다. 일반 Flink 애플리케이션을 만들고 1.19를 고르는 것은 Notebook SQL 조건의 대체가 아닙니다. [S2]

S3 폴더와 빈 버킷. 폴더는 실제 디렉터리가 아니라 객체 키의 접두사입니다. Create folder로 만든 0바이트 객체도 객체입니다. Workflow 제출 전에는 input/processed/error/ 폴더 표시용 객체까지 비워야 합니다. 업로드와 처리 후에는 키에 따라 폴더가 다시 표시됩니다.

Workflow의 행 오류와 실행 실패. 제공 test.csv는 정상 5행, 오류 4행입니다. 오류 행이 있어도 처리 함수가 statusCode 200을 반환하므로 실행은 성공하고 processed/에 CSV, error/에 행별 JSON이 함께 생기는 것이 맞습니다.

Legacy 실행 제약. 지금 root와 stub는 Linux ARM64, MSK producer는 Linux x86_64입니다. root는 NFS 공유 파일 시스템을 검사하고, stub의 고정 이름 단서는 stub.wsi.local입니다. 단순 폴더 생성이나 임의 도메인 설정으로 대체하면 실행이 실패할 수 있습니다. ARM64 ECS, EFS, DNS 서비스 검색을 사용합니다.

Aurora MySQL 8.4. 실제 제공되는 엔진입니다. 8.0 호환 Aurora 3.x로 바꾸지 마세요. 다만 8.4는 TLS 연결 강제가 기본이며 지금 ingestion은 TLS 옵션 없이 접속합니다. 앱 수정 금지 조건과의 연결 방법을 Legacy 장에서 설명합니다. [S3, S4]

실습 중 값을 기록하는 방법

메모장에 과제별 제목을 만들고 아래 값을 저장하세요. 비밀번호는 공유 문서나 화면 캡처에 포함하지 마세요.

기록할 값	찾는 위치	사용 시점
12자리 계정 ID	콘솔 우측 위 계정 메뉴	IAM 정책 ARN
본인 등번호	대회 지급 정보	세 개의 버킷 이름
VPC와 서브넷 ID	VPC 상세 화면	EC2, ALB, MSK, ECS 생성
보안 그룹 ID	EC2 또는 VPC 보안 그룹	소스 보안 그룹 지정
함수 ARN과 역할 이름	Lambda 구성 및 IAM	트리거, Step Functions
ALB DNS 이름	EC2 → Load balancers	curl 검증
MSK Cluster ARN	MSK 클러스터 상세	IAM의 cluster UUID 포함
IAM bootstrap brokers	MSK → View client information	producer-consumer-토픽 생성
DB Writer endpoint	RDS → Connectivity and security	stub와 ingestion
Distribution ID와 domain	CloudFront 배포 상세	무효화와 접속 테스트

REPLACE_ACCOUNT_ID는 계정 ID, REPLACE_STUDENT_BUCKET은 완성한 버킷 이름으로 바꿉니다. REPLACE_가 남은 파일을 저장하지 마세요. ARN의 리전도 함께 확인합니다. 샘플 명령의 <값>은 꺾쇠까지 지우고 본인 값으로 바꿔야 합니다.

연습 순서와 4시간 운영 전략

처음 배우는 날에는 4시간 안에 끝내는 것보다 과제별 성공 상태를 만드는 것이 우선입니다. 한 과제씩 진행해 설정과 오류를 기록하세요. 두 번째 연습부터 생성 대기 시간을 겹칩니다.

실전에서는 MSK·Aurora·CloudFront·EFS·캐시처럼 생성이 오래 걸리는 리소스를 먼저 요청합니다. 기다리는 동안 Workflow 코드를 넣고 Analytics 앱을 올립니다. 마지막 20~30분은 기능 점검과 Workflow 데이터 비우기를 위해 확보합니다. 리전별 브라우저 탭에 과제 이름을 붙여 잘못된 리전에 만드는 실수를 줄이세요.

처음부터 모든 리소스의 세부 최적화를 하지는 않습니다. 먼저 데이터가 끝까지 흐르게 만든 다음 제한 시간·권한·재부팅 자동 실행·태그를 검증합니다. 시험용 부하 도구는 종료하지만, MSK producer처럼 과제가 지속 실행을 요구하는 서비스는 남겨야 합니다.

1 공통 콘솔 조작과 네트워크

1단계 콘솔 화면과 입력 위치 익히기

1. AWS 콘솔에 로그인하고 상단 검색창에서 서비스 이름을 검색합니다. 예를 들어 S3를 입력한 뒤 검색 결과의 S3를 누릅니다.
2. 오른쪽 위 리전을 과제 표에 맞게 선택합니다. 서비스 화면마다 리전을 다시 확인합니다.
3. Create 또는 생성 버튼을 누르고 입력표의 값을 옮깁니다. 나머지 옵션은 이 책에서 바꾸라고 한 것만 변경합니다.
4. 생성 직후에는 상태가 Creating 또는 Pending일 수 있습니다. 새로고침해서 Available·Active·Running을 확인합니다.
5. 이름과 별개로 Tags가 있으면 Key에 Name, Value에 요구 이름을 추가합니다. Name tag 포함 조건이 있는 Legacy는 빠뜨리지 않습니다.

화면의 메뉴 번역이나 배치가 달라도 서비스 이름과 설정 이름으로 찾으세요. 이 해설서의 경로에서 화살표는 순서대로 클릭한다는 의미입니다. 실제 로그인한 콘솔의 스크린샷을 재현한 문서는 아니므로 버튼의 위치보다 이름을 기준으로 봅니다.

2단계 IAM 역할 만들기

작업 위치: IAM → Roles → Create role.

1. Trusted entity type에서 AWS service를 선택합니다.
2. Use case에서 해당 역할을 사용할 서비스, 예를 들어 Lambda 또는 EC2를 고릅니다.
3. Lambda에는 AWSLambdaBasicExecutionRole을 연결합니다. 로그가 생성되어야 에러 원인을 확인할 수 있습니다. EC2에서 Session Manager를 쓰려면 AmazonSSMManagedInstanceCore를 연결합니다.
4. Next에서 요구된 역할 이름을 입력하고 Create role을 누릅니다.
5. 역할 상세 → Permissions → Add permissions → Create inline policy → JSON을 선택합니다.
6. 동봉 iam 폴더의 해당 JSON 전체를 붙입니다. REPLACE_ 값을 모두 바꿉니다. Next에서 정책 이름을 입력하고 저장합니다.
7. Trust relationships 탭에서 서비스가 맞는지 확인합니다. 권한 정책은 무엇을 할 수 있는지, 신뢰 정책은 누가 역할을 사용할 수 있는지를 정합니다.

사용하는 서비스	신뢰 정책의 Service
Lambda	lambda.amazonaws.com
EC2	ec2.amazonaws.com
Step Functions	states.amazonaws.com
Managed Flink	kinesisanalytics.amazonaws.com
ECS Task	ecs-tasks.amazonaws.com

Step Functions와 Flink의 서비스 항목을 찾기 어려우면 Custom trust policy에서 다음 예제를 사용하고 Service 값만 표에 맞게 바꿉니다.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "states.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

완료 확인: 역할 이름이 목록에 있고 Permissions와 Trust relationships가 모두 맞습니다. AdministratorAccess로 대신하지 않습니다. IAM 정책을 저장했는데 적용이 안 된 것 같으면 잠시 후 다시 실행하고 실제 Lambda-EC2가 그 역할을 사용하는지 확인합니다.

ARN 주의: 버킷 자체 ARN에는 /*가 없고, 버킷 안 파일의 ARN에는 /경로/*가 붙습니다. DynamoDB PutItem에는 table/테이블이름 ARN을 사용합니다. HeadObject에 대응하는 IAM 권한은 s3:GetObject이며, s3:HeadObject라는 정책 작업을 쓰지 않습니다.

3단계 VPC를 직접 만들기

아래 절차는 Analytics-MSK-Legacy에서 반복합니다. 실습 편의를 위해 VPC only로 직접 만들면 문제의 이름과 CIDR을 정확히 맞출 수 있습니다.

1. VPC → Your VPCs → Create VPC → VPC only를 선택합니다.
2. Name tag와 IPv4 CIDR을 각 과제의 표대로 입력합니다. IPv6는 지정이 없으므로 선택하지 않습니다.
3. 생성한 VPC 선택 → Actions → Edit VPC settings에서 DNS resolution과 DNS hostnames를 켭니다.
4. Subnets → Create subnet → 방금 만든 VPC를 선택합니다.
5. 각 서브넷의 이름, AZ, CIDR을 입력합니다. Add new subnet으로 네 개를 만든 뒤 Create subnet을 누릅니다.
6. AZ 이름 끝의 a-b-d와 서브넷 이름을 일치시킵니다. CIDR 범위가 서로 겹치지 않는지 확인합니다.

완료 확인: 네 개의 서브넷이 같은 VPC에 있고 서로 다른 두 AZ에 배치됩니다. 이미 만들어진 default VPC를 고르지 마세요.

4단계 IGW와 공개 라우팅 구성

1. VPC → Internet gateways → Create internet gateway에서 요구 이름으로 만듭니다.
2. IGW 선택 → Actions → Attach to VPC에서 해당 VPC를 선택합니다. 생성만 하고 Attach를 빼먹으면 인터넷이 연결되지 않습니다.
3. Route tables → Create route table에서 공개 라우팅 테이블 이름과 VPC를 지정합니다.
4. 생성한 라우팅 테이블 → Routes → Edit routes → Add route를 누릅니다.
5. Destination은 0.0.0.0/0, Target은 Internet Gateway에서 방금 만든 IGW를 선택합니다. 기존 local 경로는 유지합니다.
6. Subnet associations → Edit subnet associations에서 두 public subnet을 체크하고 저장합니다.

완료 확인: 공개 라우팅 테이블의 명시적 서브넷 연결에 public 두 개가 있고 0.0.0.0/0의 대상이 igw-로 시작합니다. private subnet이 여기에 연결되지 않아야 합니다.

5단계 NAT와 비공개 라우팅 구성

1. VPC → NAT gateways → Create NAT gateway에서 이름을 입력합니다.
2. Subnet은 public-a, Connectivity type은 Public을 선택합니다. Allocate Elastic IP로 공인 주소를 지정합니다.
3. 생성 후 Available까지 기다립니다.
4. private-a용과 private-b 또는 private-d용 라우팅 테이블을 각각 만듭니다.
5. 각 테이블의 0.0.0.0/0 대상을 방금 만든 NAT Gateway로 지정합니다.
6. 각 private subnet을 자기 라우팅 테이블에 명시적으로 연결합니다.

완료 확인: public → IGW, private → NAT입니다. NAT를 private subnet에 만들면 정상적인 외부 통신 경로가 만들어지지 않습니다.

문제의 Analytics와 MSK 표에는 NAT 이름이 각각 한 개입니다. 본문 예제는 이를 따릅니다. 실제 AZ 장애까지 견디는 출구를 만들려면 AZ별 NAT와 동일 AZ 라우팅을 사용해야 하므로, 문제의 정확한 명명 조건과 고가용성 해석을 구분합니다. ALB와 MSK를 두 AZ로 만든다고 NAT까지 고가용성이 되는 것은 아닙니다.

6단계 보안 그룹 만들기

작업 위치: VPC → Security groups → Create security group.

1. 이름과 Description을 입력하고 반드시 해당 VPC를 선택합니다.
2. Inbound rules의 Add rule에서 유형, 포트, Source를 입력합니다.
3. Source에 다른 보안 그룹을 넣으려면 Custom을 선택하고 sg- ID 또는 그룹 이름을 검색합니다.
4. Outbound는 문제에서 허용한 인터넷 TCP 80-443과 해당 앱에 필요한 DB-Kafka-NFS-캐시 포트를 허용합니다.
5. 초기 생성 시 All traffic outbound를 썼다면 기능 검증 후 필요한 목적지-포트로 좁힙니다.

예시	인바운드 포트	Source
공개 ALB	TCP 80	0.0.0.0/0
주문 EC2	TCP 5000	Analytics ALB 보안 그룹
MSK Broker	TCP 9098	producer SG, consumer SG, broker SG 자신
Aurora	TCP 3306	stub SG, ingestion SG, 작업용 EC2 SG
EFS	TCP 2049	root task SG
Valkey	TCP 6379	stub task SG

보안 그룹은 응답 트래픽을 자동 허용합니다. 모든 포트를 인터넷에 열어서 해결하지 마세요. 데이터베이스 포트 3306의 Source를 0.0.0.0/0으로 만들 필요가 없습니다.

7단계 EC2를 만들고 Session Manager로 접속

1. EC2 → Instances → Launch instances를 누릅니다.

2. Name, AMI, 인스턴스 유형을 과제대로 선택합니다. Analytics와 MSK는 Amazon Linux 2023 x86_64와 t3.small을 사용합니다.
3. Network settings → Edit에서 VPC와 지정 private subnet을 선택합니다. Auto-assign public IP는 Disable입니다.
4. 기존 보안 그룹을 선택합니다. Session Manager만 사용할 경우 SSH 22 인바운드가 없어도 됩니다.
5. Advanced details → IAM instance profile에서 준비한 EC2 역할을 선택합니다.
6. Launch 후 Running과 상태 검사 통과를 기다립니다.
7. 인스턴스를 선택하고 Connect → Session Manager → Connect를 누릅니다.

터미널 창이 열리면 아래 명령을 한 줄씩 실행합니다. `\$`나 `#` 같은 프롬프트는 복사하지 않습니다.

```
whoami
uname -m
curl -I https://aws.amazon.com
```

정상 결과: 세션 사용자가 표시되고, t3.small에서는 x86_64가 나옵니다. 마지막 명령이 HTTPS 응답을 받으면 외부 통신이 가능합니다.

접속 불가: EC2 역할의 SSM 정책, SSM Agent 실행, private route의 NAT, NAT의 public route, outbound 443을 차례대로 봅니다. NAT 대신 엔드포인트를 쓰는 설계라면 ssm-ssmmessages 등의 필요한 VPC 엔드포인트와 DNS를 따로 구성해야 합니다. 이 책의 기본 경로는 NAT입니다.

8단계 파일을 private EC2로 전달

Windows 탐색기의 경로는 EC2 안에 존재하지 않습니다. 콘솔에서 만든 비공개 작업용 S3 버킷을 중간 전달 장소로 사용합니다. Workflow 채점 버킷을 전달 장소로 사용하면 제출 전 빈 버킷 조건을 어길 수 있습니다.

1. 대상 리전의 S3에서 별도 버킷을 만듭니다. 예: wsc2026-tools-계정ID-리전. 이 이름은 실습용 추천 이름입니다.
2. Block all public access를 유지합니다. 버킷 → Upload에서 필요한 원본 파일이나 zip을 올립니다.
3. EC2 역할에 그 버킷의 필요한 객체에 대한 s3:GetObject 권한을 추가합니다. KMS 암호화 버킷이라면 해당 키 Decrypt 권한도 필요합니다. 작업용 버킷은 기본 SSE-S3로 단순하게 구성할 수 있습니다.
4. Session Manager 창에서 아래와 같이 다운받습니다. REPLACE_TOOLS_BUCKET을 실제 버킷 이름으로 바꿉니다.

```
cd /tmp
aws s3 cp s3://REPLACE_TOOLS_BUCKET/day2_release_files.zip release.zip
unzip -l release.zip | head -30
unzip release.zip -d /tmp/release
find /tmp/release -name app.py -o -name config.ini
```

zip의 내부 경로에 day2_release_files가 한 번 더 들어갈 수 있습니다. find 결과의 실제 경로를 확인하고 cp 명령의 출발 경로로 씁니다. _MACOSX와 .DS_Store는 앱 파일이 아닙니다.

실습파일 zip도 같은 방법으로 올립니다. 파일이 개별적으로 필요하면 S3 Upload에서 해당 파일만 올리고 정확한 키로 받습니다. `aws s3 cp`에 AccessDenied가 나오면 서버의 인터넷보다 IAM과 객체 키를 먼저 확인합니다.

9단계 CloudShell과 서버 터미널 구분

CloudShell은 콘솔 상단의 터미널 아이콘으로 엽니다. Actions → Upload file로 내 PC의 파일을 넣고, Actions → Download file로 만든 zip을 내려받을 수 있습니다. 기본 CloudShell은 private VPC의 DB나 Kafka에 바로 접근할 수 있다고 가정하지 않습니다. DB-Kafka 관리 명령은 해당 VPC의 EC2 Session Manager에서 실행합니다.

CloudShell은 작은 Python 패키지 압축 작업에 사용됩니다. 대용량 Docker 이미지 빌드는 저장 공간과 CPU 구조 때문에 Legacy의 작업용 ARM EC2에서 수행합니다. 서비스 리소스를 만드는 곳과 앱 파일을 가공하는 곳을 구분하면 실수가 줄어듭니다.

10단계 로그를 보는 방법

Lambda: 함수 → Monitor → View CloudWatch logs → 최신 Log stream을 엽니다.

INIT_REPORT·Traceback·AccessDenied·timeout을 검색합니다.

EC2: Session Manager에서 `sudo systemctl status 서비스명 --no-pager`와 `sudo journalctl -u 서비스명 -n 80 --no-pager`를 사용합니다.

ECS: Cluster → Service → Tasks → Task 상세 → Logs를 엽니다. 실행되지 못한 태스크는 Stopped reason과 각 컨테이너의 Exit code도 봅니다.

Step Functions: State machine → Executions → 해당 실행 → 빨간 상태를 누른 뒤 Input·Output·Error를 봅니다. 전체 시스템을 한꺼번에 바꾸지 말고 실패한 단계의 권한·입력·연결을 먼저 고칩니다.

2 Workflow 상세 구축

리전: ap-southeast-1 싱가포르. **배점:** 7점. **입력:** S3의 input/test.csv. **완료 기준:** 업로드부터 60초 이내에 실행이 끝나고 정상 행 5개, 오류 JSON 4개, 처리 완료 CSV가 생깁니다.

흐름은 S3 업로드 → 트리거 Lambda → Standard Step Functions → 성적 처리 Lambda → DynamoDB와 S3입니다. S3는 파일 보관, Lambda는 계산, DynamoDB는 결과 저장, Step Functions는 처리 순서를 담당합니다.

W1 학생 성적 버킷 만들기

1. 콘솔 리전을 싱가포르로 바꿉니다. S3 → Create bucket을 누릅니다.
2. General purpose 버킷을 선택하고 이름을 wsc2026-student-score-bucket-등번호로 입력합니다. 등번호 17이라면 마지막에 17을 붙입니다. 꺾쇠는 넣지 않습니다.
3. Object ownership은 ACLs disabled, Block all public access는 유지합니다.
4. Versioning은 이 실습에서는 Disable로 둡니다. 별도 암호화 조건이 없으므로 기본 SSE-S3를 사용합니다.
5. Create bucket 후 Properties에서 AWS Region이 ap-southeast-1인지 확인합니다.
6. 지금은 폴더를 만들지 않아도 됩니다. 첫 검증 때 input/ 안에 업로드합니다.

해설: S3 키는 input/test.csv입니다. 앞에 /를 붙여 /input/test.csv로 올리면 트리거 prefix input/와 일치하지 않습니다. 버킷 이름은 전 세계에서 고유해야 하므로 이미 사용 중이라면 주어진 등번호 이름 충돌을 담당자에게 확인해야 합니다.

W2 DynamoDB 테이블 만들기

1. DynamoDB → Tables → Create table을 누릅니다.
2. Table name은 wsc2026-student-score입니다.
3. Partition key는 studentId, Type은 String입니다. 대소문자 l를 정확히 맞춥니다.
4. Sort key를 체크하고 examDate, String을 입력합니다.
5. 기본 또는 Customize settings에서 Capacity mode를 On-demand로 선택합니다. 추가 인덱스는 만들지 않습니다.
6. 생성 완료 후 Overview에서 Active와 키 두 개를 확인합니다. Explore table items는 아직 빈 화면이어야 합니다.

해설: 한 학생이 여러 시험일의 성적을 가질 수 있으므로 학생 번호와 시험일을 함께 기본 키로 사용합니다. 과목과 평균은 키가 아닌 일반 속성으로 나중에 추가됩니다.

W3 실행 역할 세 개 만들기

공통 IAM 절차를 따라 아래 역할을 생성합니다. 로그용 기본 관리 정책과 실습파일의 인라인 정책을 구분해서 추가합니다.

역할 이름	신뢰 서비스	추가할 인라인 정책 파일
wsc2026-lambda-student-role	Lambda	iam/module1-processing.json

역할 이름	신뢰 서비스	추가할 인라인 정책 파일
wsc2026-stepfunction-student-role	Step Functions	iam/module1-workflow.json
wsc2026-student-trigger-role	Lambda	iam/module1-trigger.json

Lambda 역할 두 개에는 AWSLambdaBasicExecutionRole을 연결합니다. Step Functions 역할은 이 예제에서 Lambda 호출과 S3 복사·삭제를 담당합니다. 실행 로그를 CloudWatch에 별도로 커려면 로그 전달 권한도 필요하지만 Standard의 실행 이력 조회만으로 이 장의 검증은 가능합니다.

정책 안의 REPLACE_ACCOUNT_ID와 REPLACE_STUDENT_BUCKET을 교체합니다. 트리거 역할의 대상은 아직 만들지 않았더라도 최종 이름이 정해진 stateMachine ARN으로 입력할 수 있습니다.

완료 확인: Processing 역할이 S3 input 읽기·error 쓰기·DynamoDB PutItem만 갖고, Trigger 역할은 지정 상태 머신 StartExecution만 갖습니다. 역할을 합쳐 광범위 권한을 주지 않습니다.

W4 성적 처리 Lambda 만들기

1. Lambda → Functions → Create function → Author from scratch를 선택합니다.
2. Function name은 wsc2026-student-score-function, Runtime은 Python 3.12입니다.
3. Architecture는 x86_64, Execution role은 Use an existing role에서 wsc2026-lambda-student-role을 선택합니다.
4. 생성 후 Code 탭에서 기본 lambda_function.py 대신 새 파일 index.py를 만듭니다. 실습파일 module1/index.py 전체를 붙입니다.
5. Runtime settings → Edit → Handler를 index.handler로 바꿉니다. 파일명이 index.py이고 함수명이 handler여야 합니다.
6. Deploy를 누릅니다. 코드 편집만 하고 Deploy를 누르지 않으면 이전 코드가 실행됩니다.
7. Configuration → Environment variables → Edit에서 S3_BUCKET=실제 버킷 이름, DDB_TABLE=wsc2026-student-score를 추가합니다.

8. General configuration에서 메모리 256MB, Timeout 20초를 시작값으로 설정합니다. VPC 연결은 하지 않습니다.

왜 VPC를 붙이지 않나: 이 함수는 S3와 DynamoDB API만 사용합니다. private DB가 없으므로 VPC에 넣을 이유가 없고, 잘못 연결하면 NAT 없는 함수가 AWS API에 접근하지 못합니다.

TODO 해설: 평균은 다섯 정수 점수의 합을 5로 나눕니다. DynamoDB의 Number 저장에는 Python float 대신 Decimal을 사용합니다. 평균이 90 이상이면 A, 80 이상 B, 70 이상 C, 60 이상 D, 그 외 F입니다. 큰 값부터 비교해야 95점이 D로 잘못 분류되지 않습니다.

이 코드는 지급된 validate_row·save_error·handler를 유지하고 calculate_grade와 save_student를 완성한 것입니다. 지급 코드의 error/error_타임스탬프_학생번호.json 규칙을 유지했습니다. 명세의 파일명 설명과 접두사 차이가 있으므로 채점이 파일명 전체를 강제하는 경우에는 정정 기준을 따릅니다.

W5 함수 단독으로 먼저 시험

1. S3 버킷 → Create folder로 input을 만들거나 Upload의 대상 경로를 input/으로 정합니다.
2. 지급 test.csv를 업로드합니다. 아직 자동 트리거를 연결하지 않았으므로 자동 실행되지 않습니다.
3. Lambda → Test → Create new event에서 아래 JSON을 입력합니다.

```
{"key": "input/test.csv"}
```

4. Test를 누르고 실행 결과가 statusCode 200, processed 5, errors 4인지 확인합니다.
5. DynamoDB → Explore table items에서 5개 항목과 average-grade를 확인합니다.
6. S3 → error/에서 네 개의 JSON을 확인합니다.

학생 번호	평균	등급
STU1020	96.6	A
STU1220	76.6	C
STU1203	87.2	B
STU0511	92	A
STU4444	58	F

STU2002는 INVALID_FORMAT, STU2004와 STU2001 및 학생 번호가 빈 행은 MISSING_FIELD입니다. STU2004에 점수 A도 있지만 필수 필드 검사가 먼저이므로 이름 누락이 먼저 잡힙니다. 빈 학생 번호는 지금 코드에서 파일명 끝이 빈 문자열일 수 있어도 오류 JSON은 생성됩니다.

오류 해결: Runtime.ImportModuleError면 index.py와 Handler를 확인합니다. AccessDenied면 오류 메시지의 작업과 역할 정책을 대조합니다. 400이면 환경변수와 S3 key를 확인합니다. 단독 실행에서는 CSV 이동을 하지 않으므로 input/test.csv가 남아 있는 것이 정상입니다.

W6 상태 머신 만들기

1. Step Functions → State machines → Create state machine을 누릅니다.
2. Code 또는 코드로 시작하는 방식을 선택합니다. Query language는 JSONPath를 사용합니다.
3. 실습파일 module1/workflow.asl.json을 엽니다. REPLACE_STUDENT_BUCKET과 REPLACE_ACCOUNT_ID를 모두 교체합니다.
4. JSON 전체를 Definition에 붙입니다. Validation 오류가 없는지 확인합니다.
5. Name은 wsc2026-student-score-workflow, Type은 Standard입니다.
6. Permissions에서 기존 wsc2026-stepfunction-student-role을 선택하고 생성합니다.
7. 상세 화면의 State machine ARN을 복사해 메모합니다.

상태	하는 일	중요한 설정
CheckS3File	HeadObject로 파일 존재 확인	ResultPath null로 원래 key 유지
ProcessStudentData	Lambda 호출	Payload에는 key만 전달
CheckResult	statusCode 검사	\$.result.statusCode가 200이면 성공 경로
MoveToProcessed	processed/로 CopyObject	복사가 성공해야 삭제 단계 진행

상태	하는 일	중요한 설정
DeleteProcessedInput	원본 DeleteObject	최종 End로 이동
MoveToError	error/로 CopyObject	실패 파일 보관
DeleteErrorInput	원본 DeleteObject	최종 Fail로 이동

해설: CopyObject 한 번으로 이동은 끝나지 않습니다. 복사 후 DeleteObject가 필요합니다. Lambda 최적화 통합의 결과에는 Payload 포장이 있으므로 ResultSelector로 필요한 필드를 꺼내 \$.result에 저장합니다. key를 통째로 덮어쓰면 다음 복사에서 경로를 찾지 못합니다.

Retry는 AWS 서비스 호출의 일시적 오류에만 1초, 2초 간격으로 설정했습니다. 함수가 정상 반환한 statusCode 400은 Retry의 예외가 아니라 Choice에서 판단하는 값입니다. 실행 제한 55초는 무한 대기를 줄이는 시작값이며, 업로드 이벤트 전달까지 포함한 60초 충족은 실제로 측정해야 합니다.

동봉 ASL은 채점 경로 input/test.csv처럼 한 단계의 단순 ASCII 파일명을 전제로 합니다. input/a/b.csv나 공백·한글 파일명으로 확장하려면 목적지 key 산출과 CopySource URL 인코딩을 별도 구현해야 합니다. 이 예제를 모든 S3 키에 대한 범용 해법으로 사용하지 않습니다.

W7 상태 머신 수동 실행

1. input/test.csv가 S3에 있는지 확인합니다.
2. State machine → Start execution에서 입력에 {"key": "input/test.csv"}를 넣고 실행합니다.
3. 실행 그래프가 CheckResult에서 MoveToProcessed로 진행하고 Succeeded가 되는지 확인합니다.
4. S3에서 input/test.csv는 없어지고 processed/test.csv가 있는지 확인합니다.
5. DynamoDB는 같은 PK:SK를 PutItem하면 덮어쓰므로 같은 시험 데이터를 재실행해도 10개가 아니라 5개입니다.

실패하면: 빨간 Task의 Error와 Cause를 읽습니다. CopyObject AccessDenied는 상태 머신 역할의 S3 읽기·쓰기, Lambda Invoke AccessDenied는 상태 머신 역할의 lambda:InvokeFunction, Choice 경로 오류는 Lambda 출력과 ResultSelector를 봅니다.

W8 트리거 Lambda 만들기

1. Lambda에서 wsc2026-student-score-trigger를 Python 3.12로 생성합니다. 이 트리거 이름은 해설서에서 정한 추천 이름입니다.
2. 역할은 wsc2026-student-trigger-role입니다.
3. 기본 lambda_function.py에 실습파일 module1/trigger.py 전체를 넣고 Handler는 lambda_function.lambda_handler로 둡니다.
4. Deploy 후 환경변수 STATE_MACHINE_ARN에 W6에서 복사한 ARN을 넣습니다.
5. Timeout은 10초로 설정합니다. VPC는 연결하지 않습니다.

트리거는 S3 이벤트에서 key를 꺼내 URL 디코딩하고 Step Functions를 시작합니다. S3 이벤트 전체를 상태 머신에 넘기면 {"key": "..."} 입력 형식과 달라집니다.

W9 S3 업로드 이벤트 연결

1. 트리거 함수 → Add trigger → S3를 선택합니다.
2. W1 버킷을 고르고 Event types는 All object create events를 선택합니다.
3. Prefix는 input/, Suffix는 .csv로 입력합니다.
4. 재귀 실행 안내를 확인하고 Add를 누릅니다. 여기서는 출력이 processed/error/라 input/ 필터로 재귀를 방지합니다.
5. S3 버킷 → Properties → Event notifications에서도 설정을 확인합니다.
6. Lambda → Configuration → Permissions의 Resource-based policy에서 S3가 이 함수를 호출할 권한이 추가되었는지 확인합니다.

완료 확인: 처리 함수가 아니라 트리거 함수에 S3가 연결되어 있습니다. 모든 접두사에 연결하면 오류 JSON 저장마다 다시 실행되는 잘못된 구조가 됩니다.

W10 자동 실행과 실패 경로 확인

1. 단독 검증으로 만든 S3 객체와 DynamoDB 항목을 실습 데이터로 확인한 뒤 비웁니다.
2. input/test.csv를 다시 업로드하면서 시간을 기록합니다.
3. Step Functions Executions에 새로운 실행이 나타나는지 확인합니다. Succeeded까지의 시간을 업로드 시점과 비교합니다.
4. processed/test.csv 1개, error/ JSON 4개, DynamoDB 5개를 확인합니다.
5. Lambda Test에서 {"key": "wrong/test.csv"}를 넣으면 400 반환인지 확인합니다. 이 검사는 상태 머신의 이동 동작을 검증한 것은 아닙니다.

실패 분기를 끝까지 연습하려면 별도 연습 시간에 처리 함수 DDB_TABLE을 잠시 비운 상태로 테스트 CSV를 올립니다. 그러면 함수가 400을 반환하고 error/test.csv로 이동한 뒤 Failed가 됩니다. 즉시 DDB_TABLE을 정상값으로 복원하고 정상 업로드를 다시 검증합니다. 제출 상태에 고의 오류 설정을 남기지 마세요.

S3 이벤트는 중복 전달될 수 있습니다. 이 간단한 트리거는 실행마다 새 실행을 만듭니다. 같은 파일의 중복 실행이 경쟁하면 한 실행이 먼저 이동한 뒤 다른 실행의 HeadObject가 실패할 수 있습니다. 시험에서는 한 번 올리고 완료를 확인하며, 운영용으로 확장할 때는 이벤트 ID 기반 중복 방지를 추가합니다.

W11 제출 직전 비우기

1. 자동 테스트와 업로드 반복을 멈춥니다. 진행 중인 실행이 없는지 확인합니다.
2. S3 버킷 목록에서 해당 버킷을 선택하고 Empty를 누릅니다. 버킷을 Delete하지 않습니다.
3. 객체 목록이 완전히 비었는지 확인합니다. input/ 등의 폴더 표시용 객체도 없어야 합니다.
4. DynamoDB → Explore table items → Scan으로 항목을 조회하고 시험 항목을 모두 삭제합니다. 페이지가 여러 개면 전부 확인합니다.
5. 테이블의 지연 갱신되는 Item count만 보지 말고 실제 Scan 결과가 빈지 확인합니다.
6. 테이블, Lambda, 이벤트, 상태 머신, IAM 역할은 그대로 유지합니다. 채점자가 올리는 새 CSV를 처리해야 합니다.

버킷 버전 관리를 켜다면 현재 객체뿐 아니라 과거 버전과 delete marker까지 검토합니다. 빈 상태 제출 규칙이 있으므로 이 과제에서는 불필요한 버전 관리가 정리를 복잡하게 만듭니다.

3 Real time data analytics 상세 구축

리전: ap-northeast-2 서울. 배점: 7점. 흐름: 외부 요청 → ALB 80 → private EC2 5000 → Kinesis → Flink Studio SQL.

A1 네트워크 만들기

공통 3~5단계를 아래 이름으로 수행합니다. VPC 이름은 analytics-vpc, CIDR은 10.20.0.0/16입니다. IGW 이름은 analytics-igw, NAT 이름은 analytics-ngw입니다.

서브넷	AZ	CIDR	연결할 라우팅 테이블
analytics-pub-a	ap-northeast-2a	10.20.0.0/24	analytics-pub-rtb
analytics-pub-b	ap-northeast-2b	10.20.1.0/24	analytics-pub-rtb
analytics-priv-a	ap-northeast-2a	10.20.100.0/24	analytics-priv-a-rtb
analytics-priv-b	ap-northeast-2b	10.20.101.0/24	analytics-priv-b-rtb

public 라우팅의 기본 경로는 IGW, 두 private 라우팅의 기본 경로는 NAT입니다. 생성 후 각 서브넷 상세의 Route table에서 연결 상태를 확인합니다.

A2 보안 그룹과 EC2 역할 준비

1. analytics-alb-sg를 만듭니다. Inbound TCP 80을 0.0.0.0/0에서 허용합니다. Outbound TCP 5000을 analytics-app-sg로 허용합니다.
2. analytics-app-sg를 만듭니다. Inbound TCP 5000의 Source는 analytics-alb-sg입니다. Outbound TCP 80-443을 인터넷으로 허용합니다.
3. IAM에서 wsc2026-analytics-ec2-role을 EC2 신뢰 역할로 만듭니다.
4. AmazonSSMManagedInstanceCore를 연결하고 iam/module2-ec2.json을 인라인 정책으로 추가합니다.
5. 작업용 S3에서 앱을 내려받을 경우 해당 객체 GetObject 권한도 추가합니다.

SG를 서로 참조해야 하므로 두 그룹을 우선 이름만 만들어 저장한 뒤 규칙을 편집해도 됩니다. EC2 포트 5000을 인터넷 전체에 열지 않습니다.

A3 Kinesis 스트림 만들기

1. Amazon Kinesis → Data streams → Create data stream을 누릅니다.
2. 이름은 wsc2026-order-stream입니다.
3. Capacity mode는 On-demand를 선택합니다. 세부 티어가 보이면 별도 지정이 없는 기본 On-demand 구성을 선택합니다.
4. 생성 후 Active까지 기다립니다. 스트림 ARN을 메모합니다.

해설: 앱은 주문 한 건마다 PutRecord를 호출합니다. 서버 IAM에 PutRecord 권한을 주고 앱 환경변수의 스트림 이름과 리전을 일치시켜야 합니다. 이 문제는 Kinesis Data Firehose 배달 스트림 생성 문제가 아닙니다.

A4 private EC2 생성

1. EC2 → Launch instances에서 이름을 wsc2026-analytics-ec2로 입력합니다.
2. AMI는 Amazon Linux 2023 x86_64, Instance type은 t3.small입니다.
3. analytics-vpc, analytics-priv-a, Public IP Disable, analytics-app-sg를 선택합니다.
4. Advanced details에서 IAM instance profile을 wsc2026-analytics-ec2-role로 지정합니다.
5. 생성 후 Connect → Session Manager로 접속합니다.

완료 확인: Public IPv4 address가 없고 Subnet이 private-a입니다. SSM 접속이 안 되면 A1의 NAT부터 확인합니다.

A5 파일 배치와 Python 환경 설치

공통 파일 전달 절차로 module2의 app.py, requirements.txt를 /tmp에 내려받습니다. 다음 명령은 EC2의 Session Manager에서 실행합니다. /tmp/app.py와 /tmp/requirements.txt가 실제로 존재하는지 먼저 확인합니다.

```
ls -l /tmp/app.py /tmp/requirements.txt
sudo dnf install -y python3.12 python3.12-pip
sudo mkdir -p /opt/app
sudo cp /tmp/app.py /tmp/requirements.txt /opt/app/
sudo chown -R ec2-user:ec2-user /opt/app
sudo -u ec2-user python3.12 -m venv /opt/app/venv
sudo -u ec2-user /opt/app/venv/bin/pip install -r /opt/app/requirements.txt
```

정상 결과: 패키지 설치가 성공합니다. dnf에서 python3.12를 찾지 못하면 AMI와 저장소를 확인하고 AL2023 패키지 업데이트 후 다시 조회합니다. 무조건 python3로 바꾸면 지금 환경 Python 3.12와 달라질 수 있습니다.

A6 앱 환경변수와 systemd 등록

Session Manager에서 아래 명령을 붙입니다. EOF 줄까지 포함해 한 블록씩 실행합니다. heredoc은 EOF가 나올 때까지의 내용을 파일에 저장하는 문법입니다.

```
sudo tee /etc/app.env >/dev/null <<'EOF'
STREAM_NAME=wsc2026-order-stream
AWS_REGION=ap-northeast-2
EOF
```

다음 서비스 파일은 실습파일 module2/app.service와 같습니다.

```
sudo tee /etc/systemd/system/app.service >/dev/null <<'EOF'
[Unit]
Description=Order stream application
After=network-online.target
Wants=network-online.target

[Service]
User=ec2-user
WorkingDirectory=/opt/app
EnvironmentFile=/etc/app.env
ExecStart=/opt/app/venv/bin/gunicorn --workers 2 --bind 0.0.0.0:5000 app:app
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable --now app
sudo systemctl status app --no-pager
curl -i http://127.0.0.1:5000/health
```

정상 결과: active (running), HTTP 200, {"status":"healthy"}입니다. gunicorn의 app:app은 app.py 파일 안의 Flask 객체 app을 뜻합니다. 0.0.0.0에 바인딩해야 ALB가 접근할 수 있습니다.

실패 시: `sudo journalctl -u app -n 80 --no-pager`를 봅니다. STREAM_NAME 누락이면 env 파일, ModuleNotFoundError면 venv 패키지, 203/EXEC면 ExecStart 경로를 확인합니다. `python app.py &`만 실행하면 재부팅 자동 실행 채점 조건을 만족하지 못합니다.

A7 타깃 그룹과 ALB 생성

1. EC2 → Target groups → Create target group에서 Target type을 Instances로 선택합니다.
2. Name은 wsc2026-analytics-tg, Protocol HTTP, Port 5000, VPC analytics-vpc입니다.
3. Health check path를 /health, Success codes를 200으로 설정합니다.
4. Next에서 wsc2026-analytics-ec2를 선택하고 Include as pending below 후 Create를 누릅니다.
5. Load balancers → Create → Application Load Balancer를 선택합니다.
6. Name은 wsc2026-analytics-alb, Scheme은 Internet-facing, IP type은 IPv4입니다.
7. Network mapping에 analytics-vpc와 public-a-public-b를 선택합니다.
8. SG는 analytics-alb-sg, Listener는 HTTP 80, Default action은 wsc2026-analytics-tg로 지정합니다.
9. 생성 완료 후 타깃 그룹의 Targets에서 Healthy를 확인합니다.

해설: ALB는 공개 서브넷에 있지만 서버는 내부 서브넷에 있습니다. 사용자는 ALB에 접속하고 ALB만 서버의 5000 포트에 접근합니다. Health check가 기본 /로 남으면 앱이 404를 반환해 Unhealthy가 될 수 있습니다.

A8 주문 데이터가 실제로 들어가는지 확인

CloudShell 또는 서버 터미널에서 ALB_DNS를 실제 DNS 이름으로 바꿉니다. HTTPS 인증서를 만들지 않았으므로 여기서는 http입니다.

```
curl -i http://ALB_DNS/health
curl -i -X POST http://ALB_DNS/order
curl -i -X POST http://ALB_DNS/orders/generate
```

정상 결과: health는 200, 주문 생성은 201, 일괄 생성은 generated 10입니다. 브라우저 주소창만 열면 GET이므로 POST 주문 API를 시험할 수 없습니다.

Kinesis → 스트림 → Data viewer에서 shard와 시작 위치를 선택해 JSON 레코드를 확인합니다. Data viewer 메뉴가 보이지 않으면 Monitoring의 IncomingRecords 증가와 Flink SELECT 결과로 교차 확인합니다. order_id-product_name-price-quantity-event_time이 있어야 합니다.

중요: 제공 앱의 event_time은 UTC를 시간대 없는 문자열로 기록합니다. SQL 환경은 UTC로 맞춥니다. 서울 리전에 만들었다고 앱 시간이 자동으로 KST가 되는 것은 아닙니다.

A9 Studio Notebook 생성

버전 확인 먼저: 원문 1.19와 Studio 조합의 충돌은 앞 장을 참고합니다. 지원 가능한 ZEPPELIN-FLINK-3_0 및 Flink 1.15 계열 조합으로 연습하되, 이를 원문 1.19 충족이라고 표시하지 않습니다. [S2, S5]

1. AWS Glue → Data Catalog → Databases → Add database에서 analytics_db를 만듭니다. 이름은 해설서 추천값입니다.

2. IAM에서 wsc2026-analytics-flink-role을 kinesisanalytics.amazonaws.com 신뢰 역할로 만듭니다.
3. iam/module2-flink-read.json으로 지정 스트림 읽기 권한을 추가합니다. CloudWatch → Log groups에서 /aws/kinesis-analytics/wsc2026-analytics-flink를 미리 만들고, iam/module2-flink-catalog.json으로 analytics_db와 그 로그 그룹에 대한 권한을 추가합니다. Studio 화면에서 다른 로그 그룹을 지정했다면 정책 ARN도 실제 그룹으로 맞춥니다. 임의의 KinesisFullAccess로 끝내지 않습니다.
4. Amazon Managed Service for Apache Flink → Studio notebooks → Create Studio notebook을 누릅니다. SQL applications나 일반 streaming application 메뉴와 구분합니다.
5. 이름은 wsc2026-analytics-flink입니다. Custom settings에서 런타임 표시값, 기존 IAM 역할, Glue database analytics_db를 선택합니다.
6. 로그를 활성화하고 생성합니다. 기본 공개 서비스 엔드포인트로 Kinesis에 접근할 수 있으면 불필요한 VPC 연결은 추가하지 않습니다.
7. Run 또는 Start를 누르고 Running이 되면 Open in Apache Zeppelin을 누릅니다.

Glue 권한 범위: 선택한 데이터베이스 조회에는 glue:GetDatabase-GetDatabases와 필요한 catalog/database ARN, 테이블 조회에는 GetTable-GetTables-GetPartitions 등이 사용됩니다. 영구 CREATE TABLE을 할 때는 CreateTable-UpdateTable-DeleteTable을 그 DB의 table/* 범위에 추가합니다. 본문 DDL은 TEMPORARY TABLE을 사용하지만 Studio의 catalog 초기화 권한까지 없어도 된다는 의미는 아닙니다.

A10 SQL 셀 작성

1. Zeppelin에서 새 Note를 만듭니다. 이름은 order-analysis로 지정합니다.
2. 실습파일 module2/notebook.sql을 엽니다. %flink.sql로 시작하는 덩어리마다 별도 셀에 넣습니다.
3. 첫 셀에서 시간대를 UTC로 설정합니다.
4. 두 번째 셀에서 order_stream 임시 테이블을 생성합니다. event_time은 STRING이 아닌 TIMESTAMP(3)입니다.
5. 세 번째 셀에서 문제의 최근 1분 쿼리를 실행합니다.
6. 네 번째 셀에서 상품별 누적 매출 쿼리를 실행합니다. 집계 결과가 갱신되므로 type=update를 사용합니다.

```
%flink.sql(type=update)
SELECT product_name, SUM(price * quantity) AS total_revenue
FROM order_stream
GROUP BY product_name;
```

정상 결과: 주문을 추가하면 상품별 매출이 늘어납니다. Laptop 한 건의 price가 1200000, quantity가 2이면 해당 레코드 매출은 2400000입니다.

connector=kinesis, stream 이름, aws.region을 정확히 입력합니다. TRIM_HORIZON은 남아 있는 초기 데이터부터 읽고 LATEST는 새 데이터부터 읽습니다. 테이블을 만든 뒤 데이터가 없으면 A8의 주문 생성 요청을 다시 보냅니다. 공식 SQL 예제의 문법을 바탕으로 입력 JSON 스키마에 맞춘 DDL입니다. [S6]

A11 최근 1분 쿼리를 정확히 이해하기

원문의 WHERE event_time > CURRENT_TIMESTAMP - INTERVAL '1' MINUTE`는 스트리밍 행이 처리될 때 조건을 평가합니다. 시간이 지났다는 이유만으로 이미 집계한 모든 과거 행이 자동 차감되는 완전한 이동 창이라고 보장할 수 없습니다. 시험에서는 원문 SQL 실행을 먼저 확인하고, 실무용 이동 집계가 필요하다면 watermark를 둔 HOP 등 시간 창을 별도 설계합니다.

이 한계를 감추려고 원문 쿼리를 임의의 다른 쿼리로 바꾸지 않습니다. 원문 실행 결과와 실제 최근 60초 통계의 의미는 별도로 확인해야 합니다. 재실행 후 오래된 데이터만 있으면 0이 나올 수 있으므로 현재 시각에 생성한 주문으로 시험합니다.

A12 최종 검증

1. EC2 → Reboot instance로 재부팅합니다. Running만으로 끝내지 말고 SSM에서 `systemctl is-enabled app``과 `systemctl is-active app``을 확인합니다.
2. ALB /health가 다시 200인지 확인합니다.
3. 주문 한 건 생성 후 Kinesis 레코드와 SQL 결과를 확인합니다.
4. EC2 public IP 없음, SG 5000의 Source가 ALB SG인지 확인합니다.
5. 불필요한 주문 반복 생성 루프는 종료합니다. app 서비스는 활성 상태로 남깁니다.

주요 장애: ALB 503이면 Healthy target이 없는지, 504면 SG-포트-프로세스를 봅니다. /health만 성공하고 /order가 500이면 EC2 Kinesis 권한-스트림-리전을 봅니다. SQL에 Table not found면 같은 Notebook 세션에서 DDL을 먼저 실행했는지 확인합니다. No factory for connector kinesis면 Studio 런타임과 포함 커넥터를 확인하고 일반 Flink 런타임 선택 오류를 의심합니다.

4 MSK 상세 구축

리전: ap-northeast-1 도쿄. **배점:** 7점. producer → raw topic → sensor consumer → 정상은 DynamoDB, 이상은 alert topic → alert consumer → SNS와 S3의 순서입니다.

사진 본문에 SNS 이름이 없지만 지금 module3/lambda.md는 SNS 발송을 요구합니다. 따라서 wsc2026-sensor-alerts 라는 추천 이름의 Standard SNS topic을 추가합니다. Kafka의 alert topic과 SNS topic은 서로 다른 서비스입니다.

M1 네트워크와 보안 그룹 만들기

VPC 이름은 msk-vpc, CIDR은 192.168.0.0/16입니다. IGW는 msk-igw, NAT는 msk-ngw입니다. 공통 네트워크 절차를 사용합니다.

서브넷	AZ	CIDR	라우팅 테이블
msk-pub-a	ap-northeast-1a	192.168.0.0/24	msk-pub-rtb
msk-pub-d	ap-northeast-1d	192.168.1.0/24	msk-pub-rtb
msk-priv-a	ap-northeast-1a	192.168.10.0/24	msk-priv-a-rtb
msk-priv-d	ap-northeast-1d	192.168.11.0/24	msk-priv-d-rtb

다음 보안 그룹 세 개를 생성합니다. 브로커 자기 자신을 Source로 넣는 규칙은 Lambda MSK poller 연결에 중요합니다.

그룹	인바운드	아웃바운드
msk-broker-sg	9098 from producer SG, consumer SG, 자기 자신	broker 간 필요 통신 및 443
msk-producer-sg	SSM만 쓰면 별도 인바운드 없음	9098 to broker SG, 80-443
msk-consumer-sg	별도 인바운드 없음	9098 to broker SG, 443

클러스터 내부 통신을 위해 broker SG 자기 참조 All traffic 규칙을 사용할 수 있습니다. 외부 전체에 All traffic을 여는 것과 구분하세요. MSK가 클러스터를 구성할 때 요구하는 broker 간 통신을 차단하지 않습니다.

M2 MSK 클러스터 생성

1. Amazon MSK → Clusters → Create cluster → Custom create를 선택합니다.
2. 이름은 wsc2026-msk-cluster입니다. Cluster type은 Provisioned입니다. Serverless는 지정 브로커 유형 조건과 다릅니다.
3. Broker 유형은 Standard 계열에서 kafka.t3.small, Kafka version은 3.6.0을 선택합니다.
4. VPC는 msk-vpc, AZ는 두 개, 서브넷은 msk-priv-a-msk-priv-d를 선택합니다. 각 AZ에 브로커 1개로 총 2개를 구성합니다.

5. SG는 msk-broker-sg를 선택합니다. 퍼블릭 액세스는 끕니다.
6. Security의 Client authentication에서 IAM access control만 켭니다. Unauthenticated-SASL/SCRAM-mTLS는 요구가 없으므로 끕니다.
7. Client-broker 통신은 TLS로 설정합니다. 나머지 저장 공간 등은 별도 지정이 없는 범위에서 기본값을 검토합니다.
8. 로그를 CloudWatch에 보내도록 설정하면 장애 분석이 쉽습니다. Create cluster 후 Active까지 기다립니다.

버전 안내: 3.6.0의 신규 생성 가능 여부는 AWS 지원 정책과 계정·리전에 따라 달라질 수 있습니다. 목록에서 없어졌다면 최신 버전으로 몰래 대체하고 정답으로 처리하지 않습니다. 대회 허용 대체 버전을 확인해야 합니다. 이 문서는 3.6.0이 모든 계정에서 현재 생성 가능하다고 보장하지 않습니다. [S7]

M3 브로커 주소와 ARN 기록

1. 클러스터 상세의 Cluster ARN을 복사합니다.
2. View client information을 열어 Private endpoint의 IAM bootstrap brokers를 복사합니다.
3. 주소 끝의 포트가 9098인지 확인합니다. TLS 9094 주소와 혼동하지 마세요.
4. IAM 파일의 REPLACE_CLUSTER_UUID는 Cluster ARN의 클러스터 이름 뒤 마지막 부분 전체로 교체합니다. 마지막에 -1 같은 문자가 붙으면 포함합니다.

예를 들어 Cluster ARN이 `arn:aws:kafka:ap-northeast-1:123456789012:cluster/wsc2026-msk-cluster/abc-123-1``이면 topic ARN은 `....topic/wsc2026-msk-cluster/abc-123-1/wsc2026-sensor-raw``입니다. cluster ARN 뒤에 topic 이름만 붙이는 형태가 아닙니다.

M4 producer 역할과 EC2 생성

1. IAM에서 wsc2026-msk-ec2-role을 EC2 신뢰 역할로 만듭니다.
2. AmazonSSMManagedInstanceCore와 iam/module3-producer.json을 연결합니다.
3. 토픽을 이 서버에서 만들기 위해 iam/module3-topic-admin-temporary.json을 임시 인라인 정책으로 추가합니다. 생성 후 제거합니다.
4. 원본 module3/app과 Kafka 도구를 받을 작업용 S3 GetObject 권한을 추가합니다.
5. EC2를 Name=wsc2026-sensor-producer, t3.small, Amazon Linux 2023 x86_64로 만듭니다.
6. msk-vpc, msk-priv-a, public IP 없음, msk-producer-sg, 위 IAM 역할을 선택합니다.
7. Session Manager로 접속합니다. `uname -m`` 결과 x86_64가 맞는지 확인합니다.

해설: 지금 producer는 Go 바이너리라 Python 설치로 실행하지 않습니다. `chmod +x`` 후 직접 실행합니다. module5의 ARM64 바이너리와 아키텍처가 다릅니다.

M5 Kafka 관리 도구 준비

토픽 관리에는 Kafka 클라이언트가 필요합니다. 콘솔에서 토픽 생성·파티션 설정 기능을 제공하는 계정이라면 동일 값을 입력할 수 있지만, 아래는 콘솔의 Session Manager에서 재현할 수 있는 경로입니다.

1. 작업용 PC 브라우저에서 Apache Kafka 공식 archive의 kafka_2.13-3.6.0.tgz를 받습니다. 공식 경로는 부록 S8입니다.

2. AWS의 aws-msk-iam-auth 공식 GitHub Releases에서 aws-msk-iam-auth-버전-all.jar` 파일을 받습니다. 소스 zip이 아니라 all.jar입니다. 선택한 버전을 메모합니다. [S9]
3. 두 파일을 도쿄 작업용 S3 버킷에 업로드합니다.
4. EC2 세션에서 두 파일을 /tmp로 다운로드합니다. 아래 JAR 이름은 실제 파일명으로 바꿉니다.

```
sudo dnf install -y java-17-amazon-corretto-headless tar gzip
sudo mkdir -p /opt/kafka /opt/msk-auth
sudo tar -xzf /tmp/kafka_2.13-3.6.0.tgz --strip-components=1 -C /opt/kafka
sudo cp /tmp/aws-msk-iam-auth-REPLACE_VERSION-all.jar /opt/msk-auth/iam-auth.jar
export CLASSPATH=/opt/msk-auth/iam-auth.jar
java -version
```

Session Manager 연결을 새로 열면 export 값이 사라질 수 있으므로 토픽 명령 전에 CLASSPATH를 다시 지정합니다. 토픽 생성용 관리자과 실제 producer의 권한을 구분합니다.

M6 토픽 두 개 생성

실습파일 module3/client.properties 내용을 /tmp/client.properties로 저장합니다. 내용은 다음 네 줄입니다.

```
security.protocol=SASL_SSL
sasl.mechanism=AWS_MSK_IAM
sasl.jaas.config=software.amazon.msk.auth.iam.IAMLoginModule required;
sasl.client.callback.handler.class=software.amazon.msk.auth.iam.IAMClientCallbackHandler
```

다음은 EC2 Session Manager에서 실행합니다. BROKERS 값은 쉼표로 이어진 IAM bootstrap brokers 전체입니다.

```
export AWS_REGION=ap-northeast-1
export CLASSPATH=/opt/msk-auth/iam-auth.jar
export BROKERS='REPLACE_IAM_BOOTSTRAP_SERVERS'
/opt/kafka/bin/kafka-topics.sh --bootstrap-server "$BROKERS" \
  --command-config /tmp/client.properties --create \
  --topic wsc2026-sensor-raw --partitions 3 --replication-factor 2
/opt/kafka/bin/kafka-topics.sh --bootstrap-server "$BROKERS" \
  --command-config /tmp/client.properties --create \
  --topic wsc2026-sensor-alert --partitions 1 --replication-factor 2
/opt/kafka/bin/kafka-topics.sh --bootstrap-server "$BROKERS" \
  --command-config /tmp/client.properties --describe \
  --topic wsc2026-sensor-raw
```

정상 결과: raw의 PartitionCount는 3, ReplicationFactor는 2이고 각 파티션 ISR이 두 브로커를 포함합니다. alert도 describe해서 1과 2를 확인합니다. 토픽이 이미 있으면 무작정 지우지 말고 describe부터 합니다.

오류 해결: IAMLoginModule을 찾지 못하면 JAR-CLASSPATH, Timeout이면 private route와 9098 SG, Topic authorization failed면 kafka-cluster:CreateTopic의 topic ARN을 확인합니다. IAM 정책의 kafka:*만으로 Kafka 데이터 읽기·쓰기 권한이 생기지 않습니다.

M7 DynamoDB와 S3와 SNS 생성

1. DynamoDB → Create table: 이름 wsc2026-sensor-data, PK sensorId String, SK timestamp String, On-demand로 생성합니다.
2. S3 → Create bucket: wsc2026-sensor-alert-bucket-등번호, 도쿄 리전, Block public access 유지로 생성합니다. Access Point를 만들지 않습니다.
3. SNS → Topics → Create topic: Standard, Name wsc2026-sensor-alerts로 생성하고 ARN을 기록합니다.
4. 본인에게 알림 수신을 연습하려면 Create subscription → Email → 본인 주소를 넣고 받은 확인 메일에서 Confirm 합니다. 구독 확인 전에는 메일이 오지 않습니다.

DynamoDB 속성	타입	저장 내용
sensorId	String	PK
timestamp	String	KST ISO 8601, 예 2026-05-30T23:00:00+09:00
humidity	Number	습도
location	String	위치
status	String	NORMAL
temperature	String	원문 표에 맞춘 문자열 온도

테이블을 만들 때 모든 속성을 미리 정의하는 화면은 없습니다. 키만 정하고 PutItem할 때 일반 속성이 생깁니다. temperature를 Number로 바꾸고 싶은 유혹이 있어도 원문 표는 String입니다.

버킷 확인이 필요하다면 도교 CloudShell에서 `aws s3api head-bucket --bucket 실제버킷이름 --region ap-northeast-1``을 사용합니다. 최신 CLI에서 AccessPointAlias:false 표시를 확인합니다. 버킷 이름 대신 별도의 access point alias를 전달하지 않습니다.

M8 Lambda 역할 만들기

1. IAM에서 wsc2026-msk-lambda-role을 Lambda 신뢰 역할로 생성합니다.
2. AWSLambdaBasicExecutionRole을 연결합니다.
3. iam/module3-lambda.json의 계정 ID-cluster UUID-alert 버킷 이름을 교체하여 인라인 정책으로 추가합니다.
4. 정책의 소비자 그룹 이름 wsc2026-raw-consumer와 wsc2026-alert-consumer를 메모합니다. M11에서 동일하게 입력해야 합니다.

이 정책은 두 함수가 공유하는 요구 역할에 필요한 권한의 합집합입니다. producer의 raw 쓰기와 sensor consumer의 alert 쓰기를 혼동하지 않습니다. Lambda의 MSK poller에는 클러스터 조회, ENI 관리, Kafka topic-group 권한이 필요하며, 함수 코드가 alert topic에 직접 쓰는 권한도 별개로 필요합니다. [S10]

M9 sensor consumer 배포 zip 만들기

Lambda 기본 편집기에 코드를 붙이는 것만으로 kafka-python과 IAM signer가 설치되지 않습니다. CloudShell에서 의존성과 index.py를 한 zip에 넣습니다.

1. CloudShell → Actions → Upload file로 실습파일 module3/sensor_consumer/index.py와 requirements.txt를 업로드합니다.
2. 빈 작업 디렉터리를 만들고 의존성을 설치합니다. 아래 명령은 CloudShell에서 실행하며 EC2 producer 바이너리를 실행하는 명령과 다릅니다.

```
mkdir -p ~/sensor-build/package
cp ~/index.py ~/sensor-build/package/index.py
python3 -m pip install --target ~/sensor-build/package \
  --python-version 3.14 --only-binary=:all: \
  -r ~/requirements.txt
cd ~/sensor-build/package
zip -r ../sensor-consumer.zip .
unzip -l ../sensor-consumer.zip | head -20
```

3. zip의 최상위에 index.py가 있는지 확인합니다. package/index.py로 한 겹 감싸져 있으면 Handler가 찾지 못합니다.
4. Actions → Download file에서 `/home/cloudshell-user/sensor-build/sensor-consumer.zip`을 입력합니다. 홈 경로가 다르면 `pwd`로 실제 경로를 확인합니다.
5. Lambda → Create function: wsc2026-sensor-consumer, Python 3.14, x86_64, 기존 wsc2026-msk-lambda-role을 선택합니다.
6. Code → Upload from → .zip file로 업로드하고 Handler를 index.handler로 설정합니다.

requirements.txt의 버전 범위는 학습 예제입니다. 재현 가능한 환경을 보관하려면 성공한 설치 버전을 기록하고 이후 연습에 고정합니다. 이 책은 AWS 실배포에서 해당 라이브러리 조합을 검증했다고 주장하지 않습니다. Python 3.14 런타임 자체는 AWS에서 지원합니다. [S11]

M10 Lambda 두 개 설정

sensor consumer의 Configuration → Environment variables에 다음 값을 추가합니다.

변수	값
DDB_TABLE	wsc2026-sensor-data
ALERT_TOPIC	wsc2026-sensor-alert
BOOTSTRAP_SERVER	IAM bootstrap brokers 전체

producer의 변수는 BOOTSTRAP_SERVERS로 끝에 S가 있고 소비자는 BOOTSTRAP_SERVER입니다. Lambda의 AWS_REGION은 AWS가 자동 제공하므로 수동 등록하지 않습니다.

1. General configuration에서 Memory 512MB, Timeout 60초를 시작값으로 설정합니다.
2. Configuration → VPC → Edit에서 msk-vpc, private 두 개, msk-consumer-sg를 선택합니다. 이 함수는 코드 안에서 alert topic에 직접 연결하므로 VPC 경로가 필요합니다.
3. 두 번째 함수 wsc2026-sensor-alert-consumer를 Python 3.14, 같은 역할로 생성합니다.
4. 실습파일 module3/alert_consumer/index.py를 Code의 index.py에 넣고 Handler index.handler, Deploy를 설정합니다. 이 함수는 기본 boto3만 사용합니다.
5. 환경변수 SNS_TOPIC_ARN=만든 SNS ARN, S3_BUCKET=alert 버킷 이름을 설정합니다.
6. 두 번째 함수는 코드에서 S3·SNS만 호출하므로 이 예제에서는 함수 자체 VPC 연결을 생략해도 됩니다. MSK 이벤트 소스 매핑은 별도 경로로 Kafka를 읽습니다.

로직 해설: temperature가 80 초과 또는 10 미만, humidity가 90 초과 또는 20 미만이면 ALERT입니다. 80·10·90·20은 경계 안이므로 다른 조건이 정상이라면 NORMAL입니다. 정상만 DynamoDB에 저장하고 이상은 Kafka alert에 기록합니다. alert consumer가 SNS를 발송하고 alert/센서ID/날짜/시각.json으로 S3에 저장합니다.

코드는 Base64로 전달되는 MSK value를 디코딩합니다. float 습도는 Decimal로 변환하고 온도는 String으로 저장합니다. 잘못된 JSON을 조용히 성공 처리하지 않아 배치 재시도 시 확인할 수 있도록 했습니다. 운영 환경에서는 독성 메시지 처리와 별도 실패 저장 설계가 필요합니다.

M11 MSK 트리거 연결

1. wsc2026-sensor-consumer → Add trigger → MSK를 선택합니다.
2. 클러스터 wsc2026-msk-cluster를 선택하고 Topic에 wsc2026-sensor-raw를 넣습니다.
3. Consumer group ID는 wsc2026-raw-consumer, Starting position은 TRIM_HORIZON, Batch size는 10, Batch window는 1초를 시작값으로 설정합니다.
4. IAM 인증 클러스터를 사용하며 별도의 SCRAM secret을 연결하지 않습니다. 트리거를 활성화하고 저장합니다.
5. wsc2026-sensor-alert-consumer에도 같은 방식으로 Topic wsc2026-sensor-alert, Consumer group wsc2026-alert-consumer를 연결합니다.
6. Configuration → Triggers 또는 Event source mappings에서 상태 Enabled와 Last processing result를 확인합니다.

네트워크 해설: 이벤트 소스 매핑의 poller는 MSK 클러스터의 서브넷-보안 그룹을 사용합니다. On-demand poller는 NAT 또는 Lambda-STS 등의 엔드포인트 경로가 필요합니다. 함수 VPC를 지정했다고 poller의 네트워크까지 자동 해결되는 것은 아닙니다. 본문은 NAT가 있는 구성을 사용합니다. [S12]

IAM에 고정 consumer group ARN을 넣었는데 콘솔이 자동 생성 그룹을 쓰면 권한 오류가 납니다. 고급 설정의 그룹 ID를 다시 확인합니다. 그룹을 바꾸면 기존 offset의 의미가 달라져 데이터를 다시 읽을 수 있습니다.

M12 producer 앱을 서비스로 시작

원본 module3/app을 /tmp/app으로 내려받은 뒤 Session Manager에서 수행합니다.

```
sudo mkdir -p /opt/sensor
sudo cp /tmp/app /opt/sensor/app
sudo chmod 755 /opt/sensor/app
file /opt/sensor/app
sudo tee /etc/sensor.env >/dev/null <<'EOF'
AWS_REGION=ap-northeast-1
BOOTSTRAP_SERVERS=REPLACE_IAM_BOOTSTRAP_SERVERS
TOPIC_RAW=wsc2026-sensor-raw
EOF
```

REPLACE_IAM_BOOTSTRAP_SERVERS를 실제 주소로 바꾼 뒤 저장합니다. 실습파일 module3/sensor-producer.service를 /etc/systemd/system/sensor-producer.service로 복사합니다. 파일을 S3로 받았다면 다음 명령으로 등록합니다.

```
sudo cp /tmp/sensor-producer.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now sensor-producer
sudo systemctl status sensor-producer --no-pager
sudo journalctl -u sensor-producer -n 50 --no-pager
```

정상 결과: 센서 ID와 온습도 로그가 반복되고 Lambda 로그-DynamoDB-S3가 갱신됩니다. 원본 설명에 백그라운드 지속 실행이 적혀 있어도 프로세스가 스스로 fork하는지는 실제 서비스 상태로 확인합니다. 이 예제는 foreground 프로세스를 systemd가 관리하는 형식입니다. 실행 직후 정상 종료 후 반복 재시작한다면 --help와 로그로 데몬화 여부를 먼저 확인합니다.

M13 최종 검증과 문제 해결

1. DynamoDB에서 NORMAL 항목, String temperature, Number humidity, +09:00 timestamp를 확인합니다.
2. S3 alert/ 아래 JSON에서 status ALERT와 alert_reason을 확인합니다.

3. SNS 구독 확인 후 메시지가 수신되는지 확인합니다. 구독을 만들지 않았더라도 Publish 실패 여부는 Lambda 로그로 확인합니다.
4. Lambda 두 개의 MSK 트리거가 Enabled이고 올바른 topic을 읽는지 확인합니다.
5. producer 서버를 재부팅한 뒤 `systemctl is-active sensor-producer`를 확인합니다.
6. 토픽 생성용 임시 IAM 정책을 제거합니다. producer 기본 쓰기 권한과 SSM 권한은 유지합니다.

증상	우선 확인
producer 로그는 있으나 Lambda 호출 없음	topic 이름, 트리거 Enabled, 그룹 권한, poller NAT
소비자 실행은 되나 이상 데이터만 실패	BOOTSTRAP_SERVER, consumer VPC, alert WriteData
Unsupported float 오류	DynamoDB 숫자에 Decimal 사용 여부
정상 데이터가 아닌데 NORMAL	>와 < 경계, 문자열 숫자 변환
SNS만 실패	ARN 리전, sns:Publish, 구독 확인
알림이 중복됨	MSK 배치 재시도와 SNS 비역등성

MSK→Lambda는 재시도로 같은 레코드를 다시 처리할 수 있습니다. DynamoDB 동일 PK·SK와 S3 동일 경로는 덮어 쓸 수 있지만 SNS 알림은 중복될 수 있습니다. 이 학습용 구현은 exactly-once 알림을 보장하지 않습니다.

5 CDN Service Setup 상세 구축

리전: S3-KMS-Lambda-API Gateway는 us-east-1. CloudFront는 전역. **배점:** 3점. **목표:** 단일 CloudFront 도메인의 /static/image.png와 POST /now를 제공하고 이미지 변경을 자동 반영합니다.

C1 충돌 조건을 먼저 구분

원문에는 OAI, 고객 관리 KMS 키, CachingOptimized, 3분 이내 이미지 반영이 함께 있습니다. OAI는 SSE-KMS 읽기를 지원하지 않으므로 다음 두 경로는 동일한 정답이 아닙니다. [S1]

경로	가능한 구성	충족하지 못하거나 확인할 조건
이 장의 기술적 대안	OAC + SSE-KMS	원문 OAI 항목의 대체 인정 필요
OAI 학습 경로	OAI + SSE-S3	원문 KMS 항목 불충족

이 장은 실제 동작하는 OAC+KMS 경로를 상세 설명합니다. 대회에서는 정정 안내가 우선입니다. SSE-KMS를 유지한 채 OAI만 붙이고 버킷 정책을 계속 바꾸어도 이 제품 제한은 해결되지 않습니다. DSSE를 사용하지 말라는 조건은 단일 SSE-KMS를 사용하라는 의미로 적용합니다.

C2 KMS 고객 관리 키 생성

- 리전을 us-east-1로 바꾸고 KMS → Customer managed keys → Create key를 누릅니다.
- Key type은 Symmetric, Key usage는 Encrypt and decrypt로 선택합니다.
- Alias는 wsk2026-cdn-key 등 구분 가능한 이름으로 지정합니다. 이 키 별칭은 해설서 추천값입니다.
- Key administrators에 관리 주체를, Key users에는 업로드하는 본인 역할을 선택합니다.
- Review 후 Finish를 누르고 Key ARN을 기록합니다.

완료 확인: 키가 Enabled이고 Customer managed로 표시됩니다. aws/s3 AWS 관리 키와 다릅니다. CloudFront 복호화 권한은 배포 ARN을 얻은 뒤 C6에서 추가합니다.

C3 S3 버킷과 이미지 업로드

- S3 → Create bucket에서 wsk2026-encrypted-data-등번호를 생성합니다.
- 리전 us-east-1, Block all public access 유지, ACLs disabled를 사용합니다.
- Default encryption에서 SSE-KMS를 선택하고 C2의 고객 관리 키를 지정합니다. DSSE-KMS를 선택하지 않습니다.
- 버킷 → Create folder에서 static을 만듭니다.
- static/으로 들어가 지금 module4/image.png를 Upload합니다.
- 업로드 Properties → Metadata에 Content-Type=image/png, Cache-Control=public,max-age=0,s-maxage=86400을 설정합니다. 화면이 시스템 메타데이터 항목으로 나뉘어 있으면 해당 키를 각각 선택합니다.
- 업로드한 객체 상세의 Server-side encryption이 SSE-KMS와 맞는 키인지 확인합니다.

해설: max-age=0은 브라우저가 오래된 로컬 사본을 그대로 쓰지 않게 하고 s-maxage는 공유 캐시인 CDN의 저장 시간을 정합니다. CloudFront 캐시는 변경 이벤트의 invalidation으로 비웁니다. 버킷 기본 암호화 변경은 이미 저장된 객체를 자동 재암호화하지 않으므로 객체 자체 설정을 확인합니다.

C4 현재 시간 Lambda 생성

1. Lambda에서 wsk2026-now를 생성합니다. Runtime은 별도 원문 지정이 없으므로 Python 3.12를 예시로 사용합니다.
2. 실행 역할은 Lambda 기본 로그 권한을 가진 역할로 만듭니다. S3 읽기나 DB 권한은 필요하지 않습니다.
3. 기본 lambda_function.py에 실습파일 module4/now.py를 붙이고 Handler는 lambda_function.lambda_handler로 둡니다.
4. Deploy를 누릅니다.
5. Configuration → Environment variables에 ORIGIN_SECRET을 추가합니다. 추측하기 어려운 임의의 문자열을 사용합니다.

CloudShell에서 `openssl rand -hex 32`로 문자열을 생성할 수 있습니다. 결과를 Lambda와 CloudFront 오리진 설정에 동일하게 넣고 화면 캡처나 공용 저장소에 노출하지 않습니다.

해설: 현재 시간은 호출할 때마다 KST로 계산합니다. 문제의 2025년 예시 문자열을 하드코딩하면 안 됩니다. 함수는 x-origin-verify 헤더를 검사해 CloudFront가 전달한 요청과 비밀값 없는 API 직접 요청을 구분합니다. 이는 공유 비밀 기반의 연습용 접근 제한이며 사용자 인증 체계 전체를 대체하지 않습니다.

C5 HTTP API 생성

1. API Gateway → Create API → HTTP API의 Build를 누릅니다. REST API가 아닙니다.
2. Integration에서 Lambda를 선택하고 us-east-1의 wsk2026-now를 연결합니다.
3. API name은 wsk2026-api입니다.
4. Route의 Method를 POST, Resource path를 /now로 입력합니다.
5. Stage는 \$default, Auto-deploy를 활성화해 생성합니다.
6. API endpoint를 기록합니다. <https://문자열.execute-api.us-east-1.amazonaws.com> 형식입니다.
7. Lambda의 Resource-based policy에 이 API Gateway가 함수를 호출할 허용이 추가되었는지 확인합니다.

채점 웹페이지가 다른 도메인에서 fetch로 호출한다면 HTTP API → CORS에서 그 페이지의 Origin을 허용합니다. POST-OPTIONS와 Content-Type을 허용합니다. 채점 Origin이 제공되지 않은 실습에서는 credentials 없이 *를 임시 사용할 수 있지만, 허용 Origin을 알면 해당 값으로 좁힙니다. CORS는 브라우저 규칙이며 API 인증 수단이 아닙니다.

완료 확인: 직접 API 주소로 비밀값 없이 POST하면 403이 나오는 것이 이 예제의 의도입니다. 기능만 먼저 시험하려면 Lambda Test에서 headers에 x-origin-verify를 넣은 이벤트를 사용하고 검증 후 비밀값이 포함된 저장 이벤트를 제거합니다.

C6 CloudFront 배포와 S3 권한 설정

1. CloudFront → Distributions → Create distribution을 누릅니다.
2. S3 origin domain에서 C3 버킷의 REST endpoint를 고릅니다. s3-website 엔드포인트가 아닙니다.

3. Origin path는 비워 둡니다. 요청 자체에 /static/image.png가 있기 때문입니다.
4. Origin access에서 Origin access control settings를 선택하고 새 OAC를 만듭니다. Signing behavior는 Sign requests 또는 Always입니다.
5. 기본 behavior의 Viewer protocol policy는 Redirect HTTP to HTTPS, Allowed methods는 GET·HEAD, Cache policy는 CachingOptimized를 선택합니다.
6. Comment 또는 Description에 wsk2026-cf를 넣습니다. 별도 사용자 도메인 없이 기본 *.cloudfront.net을 사용해도 됩니다.
7. 배포를 만들고 Distribution ID와 Distribution ARN을 메모합니다.
8. 생성 화면의 Copy policy 또는 실습파일 module4/oac-bucket-policy.json을 이용해 S3 → Permissions → Bucket policy에 반영합니다. 버킷 이름과 배포 ID·계정 ID를 교체합니다.
9. KMS → 키 → Key policy에 module4/kms-statement.json의 Statement 하나를 추가합니다. 기존 관리자 정책 전체를 이 조각으로 덮어쓰지 않습니다.
10. 정책의 AWS:SourceArn이 실제 CloudFront 배포 ARN인지 확인합니다.

정상 결과: 배포가 Deployed 후 `https://배포도메인/static/image.png`가 표시됩니다. S3 객체의 직접 공개 URL은 읽히지 않아야 합니다. KMS AccessDenied와 S3 AccessDenied를 구분해 해당 정책을 봅니다.

C7 API 오리진과 동작 추가

1. CloudFront 배포 → Origins → Create origin을 누릅니다.
2. Origin domain에는 API Gateway 도메인 부분만 입력합니다. `https://와 /now`는 넣지 않습니다.
3. Protocol은 HTTPS only, Origin path는 \$default stage를 썼으므로 비워 둡니다.
4. Add custom header에서 Header name=x-origin-verify, Value=C4의 ORIGIN_SECRET을 입력합니다.
5. Origins 저장 후 Behaviors → Create behavior를 누릅니다.
6. Path pattern은 /now, Origin은 API Gateway, Viewer protocol은 HTTPS Only 또는 Redirect HTTP to HTTPS를 선택합니다. 시험 요청은 처음부터 https를 사용합니다.
7. Allowed HTTP methods는 GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE를 포함하는 옵션을 선택합니다. API에는 POST /now만 라우팅했으므로 허용된 메서드 전체가 구현되는 것은 아닙니다.
8. Cache policy는 CachingDisabled, Origin request policy는 AllViewerExceptHostHeader를 선택합니다.
9. /static/* behavior를 따로 만들어 S3 오리진과 CachingOptimized를 연결합니다. 원문에 이 패턴이 명시되어 있으므로 기본 behavior가 S3여도 패턴을 보이게 만듭니다.
10. 저장하고 배포 완료까지 기다립니다.

해설: API Gateway는 오리진 자신의 Host 헤더를 기대합니다. CloudFront 도메인을 Host로 그대로 전달하면 실패할 수 있어 AllViewerExceptHostHeader를 사용합니다. [S13] /now를 캐시하면 시간이 멈춘 것처럼 보입니다. POST 허용을 빠뜨리면 Lambda까지 도달하기 전에 요청이 거부됩니다.

C8 현재 시간과 직접 접근 차단 확인

CloudShell에서 다음 두 요청을 몇 초 간격으로 실행합니다.

```
curl -i -X POST https://CLOUDFRONT_DOMAIN/now
curl -i -X POST https://API_ID.execute-api.us-east-1.amazonaws.com/now
```

첫 번째는 200과 현재 KST 시간, 두 번째는 비밀 헤더가 없어 403이어야 합니다. 첫 번째를 재호출하면 초 값이 달라져야 합니다. 제공 예제는 일반 문자열 응답이며 Content-Type은 text/plain; charset=utf-8입니다.

403이 둘 다 나오면 ORIGIN_SECRET과 오리진 custom header 값이 같은지 확인합니다. 404면 API route와 stage, 502면 Lambda 로그·통합 응답을 봅니다. 브라우저에서만 실패하면 CORS와 preflight OPTIONS를 확인합니다.

HTTP API의 execute-api endpoint 자체를 끄면 CloudFront도 같은 endpoint를 사용하므로 함께 끄줍니다. 이 구성에서 무작정 Disable default endpoint를 선택하지 않습니다. 더 강한 제한을 위해 별도 도메인·인증 계층을 추가하려면 설계와 채점 조건을 다시 검토합니다.

C9 업로드 때 자동 무효화 Lambda 생성

1. us-east-1 Lambda에서 wsk2026-cache-invalidator를 생성합니다. 이름은 해설서 추천값입니다.
2. Python 3.12, 기본 로그 역할을 사용하고 iam/module4-invalidation.json을 그 역할에 추가합니다. 실제 배포 ID로 바꿉니다.
3. lambda_function.py에 module4/invalidate.py를 붙이고 Deploy합니다.
4. 환경변수 DISTRIBUTION_ID에 CloudFront Distribution ID를 넣습니다. 도메인이나 ARN을 넣는 칸이 아닙니다.
5. Add trigger → S3에서 CDN 버킷, All object create events, Prefix static/을 지정합니다.
6. Lambda 호출 권한과 S3 Event notification이 생성되었는지 확인합니다.

이 함수는 이벤트의 객체 key를 /static/image.png 같은 CloudFront 경로로 바꾸어 CreateInvalidation을 호출합니다. 역할에는 정확한 distribution ARN에 대한 cloudfront:CreateInvalidation만 추가합니다. 이벤트 sequencer 등을 호출 식별자로 사용해 동일 이벤트의 중복 요청을 줄입니다.

C10 브라우저 반영까지 검증

1. CloudFront에서 기존 image.png를 열어 초기 이미지를 확인합니다.
2. 다른 그림을 image.png라는 같은 이름으로 S3 static/에 덮어씁니다. Content-Type과 Cache-Control 메타데이터를 유지합니다.
3. 업로드 시각을 기록합니다. CloudFront → Invalidations에 해당 경로의 새 항목이 자동 생기는지 확인합니다.
4. Completed가 된 뒤 같은 URL에서 새 이미지가 나오는지 확인합니다.
5. 실습파일 module4/index.html을 버킷 루트 index.html로 업로드하면 <https://도메인/index.html>에서 10초마다 이미지를 다시 요청하는 확인 페이지를 사용할 수 있습니다.

중요한 구분: CloudFront invalidation은 브라우저가 이미 그려 놓은 화면을 자동 교체하지 않습니다. 열려 있는 페이지까지 갱신하려면 재요청·폴링이 필요합니다. 이 예제의 max-age=0과 확인 페이지는 그 차이를 보완합니다. 원문 3분 기준은 실제 업로드부터 브라우저 반영까지 측정해야 하며, AWS 이벤트 전달·invalidation 전파에 대해 절대 3분 보장을 주장하지 않습니다.

오류 해결: 무효화 항목이 없으면 S3 trigger와 Lambda 권한, 항목은 Completed인데 그림이 그대로면 객체를 같은 key로 바꿨는지·브라우저 캐시·요청이 다른 behavior를 타는지 확인합니다. 코드가 /static/static/image.png로 무효화하지 않는지도 봅니다.

C11 OAI를 별도로 학습하는 경우

대회 정정이 OAI와 SSE-S3로 바뀐 경우에만 해당 경로로 구성합니다. CloudFront S3 origin 편집 → Legacy access identities에서 OAI를 생성·선택하고 버킷 정책에 그 OAI의 읽기 권한을 부여합니다. 객체 암호화는 SSE-S3여야 합니다. OAI ARN은 `arn:aws:iam::cloudfront:user/CloudFront Origin Access Identity OAI_ID`` 형식입니다.

이것은 C6의 OAC+KMS 구성과 다른 분기입니다. 양쪽을 한 오리진에 동시에 적용하려고 하지 마세요. 원문 그대로의 OAI+KMS 조합은 지원 제한 때문에 완전 충족이라고 판정할 수 없습니다.

6 Legacy System Operation 상세 구축

리전: eu-central-1 프랑크푸르트. **배점:** 6점. **목표:** 지금된 root-stub를 Fargate에서 실행하고, ingestion을 Lambda로 올려 하나의 인터넷 ALB로 제공합니다. API 응답 목표는 1초 미만입니다.

앱 재개발은 금지 조건입니다. 이 장은 바이너리-ingestion 원본을 유지하고 컨테이너 포장, config.ini, DB 파라미터, DNS, 공유 저장소, 캐시 옵션으로 연결합니다. 숨겨진 동작을 추측해 새로운 앱으로 대체하지 않습니다.

L1 지금 앱에서 확인한 제약

대상	확인된 사실	반영할 구성
root와 stub	ELF 헤더의 Machine이 AArch64	ARM64 Linux Fargate와 이미지
root	shared_dir 기본값 /mnt/shgold-efs	EFS NFS 볼륨 실제 마운트
root	NFS 파일시스템 검증과 쓰기 확인	mkdir만으로 대체 불가
root	바이너리 문자열에 stub.wsi.local	Cloud Map private DNS 이름 사용
root	config.ini는 upstream.port=8081	stub 수신 포트 8081
stub	--cache-mode 실행 도움말 존재	Valkey 엔드포인트 전달
stub	캐시 cluster mode disabled 요구	단일 shard의 캐시 엔드포인트
ingestion	DB_PORT 기본값이 빈 문자열	환경변수 3306 반드시 입력
ingestion	TABLE_NAME 시작 시 검증	readings 반드시 입력
ingestion	PyMySQL 1.1.1 의존	소스와 라이브러리를 zip에 포함

고정 DNS와 캐시 옵션은 제공 바이너리의 문자열·도움말 데이터에서 확인한 단서입니다. 최종 동작은 실제 ARM64 환경에서 --help, 시작 로그, DNS 조회, 요청으로 검증합니다. 읽기 경로의 캐시가 있다고 처음 보는 모든 ID까지 1초 이내라고 미리 보장할 수는 없습니다.

L2 전체 구조와 생성 순서

ALB HTTP 80의 /ingest는 Lambda 타깃 그룹으로, 기본 경로는 root ECS 타깃 그룹으로 보냅니다. root는 stub.wsi.local:8081에 연결합니다. stub는 Aurora에서 읽고 Valkey 캐시를 사용합니다. root에는 EFS를 마운트합니다. 같은 ALB에 HTTP 8081 리스너를 추가해 stub의 /healthz를 직접 확인할 수 있게 구성합니다.

원문은 root와 stub 모두 /healthz를 사용하므로 같은 포트·같은 경로만으로 두 서비스를 구분할 수 없습니다. 별도 listener 8081은 한 ALB 조건을 유지하면서 구분하는 이 해설서의 구현 선택입니다. 채점기가 포트 80의 특정 별도 경로만 허용한다면 그 정정 조건에 맞는 ALB 규칙을 사용해야 합니다. stub 내부 조회 API는 root의 요청 서명을 요구하는 단서가 있으므로 공개 health 성공을 내부 API의 무인증 공개 요구로 해석하지 않습니다.

생성 순서는 VPC → 보안 그룹 → Aurora-EFS-Valkey → Secrets Manager → 작업용 ARM EC2 → ECR 이미지 → Cloud Map → ECS → ingestion Lambda → ALB 통합 → 성능 검증입니다.

L3 Legacy VPC와 보안 그룹

원문에 이 VPC의 이름·CIDR 지정은 없습니다. 다른 과제와 분리되는 아래 값을 예시로 사용합니다.

항목	추천 설정
VPC	shgold-vpc, 10.50.0.0/16
공개 서브넷	eu-central-1a 10.50.0.0/24, 1b 10.50.1.0/24
비공개 앱 서브넷	1a 10.50.10.0/24, 1b 10.50.11.0/24
DB 서브넷	위 비공개 두 개 또는 별도 DB 서브넷 두 개
인터넷 경로	public → IGW, private → NAT

공통 VPC 절차를 사용하고 DNS resolution-hostnames를 켭니다. 다음 SG를 같은 VPC에 생성합니다.

보안 그룹	인바운드 허용
shgold-alb-sg	80과 8081 from 인터넷
shgold-root-sg	8080 from ALB SG
shgold-stub-sg	8081 from root SG와 ALB SG
shgold-ingest-sg	별도 인바운드 없음
shgold-db-sg	3306 from stub SG, ingest SG, 작업 EC2 SG
shgold-efs-sg	2049 from root SG
shgold-cache-sg	6379 from stub SG
shgold-work-sg	SSM만 사용하면 인바운드 없음

출구는 ALB→root 8080:stub 8081, root→stub 8081-EFS 2049, stub→DB 3306:cache 6379:Secrets 443, ingestion→DB 3306:Secrets 443을 허용합니다. Fargate의 이미지 다운로드와 로그 전송에도 NAT를 통한 443이 필요합니다.

L4 Aurora MySQL 생성

1. RDS → Subnet groups → Create DB subnet group에서 shgold-db-subnets를 만듭니다. shgold-vpc와 서로 다른 AZ의 private subnet 두 개를 넣습니다.
2. RDS → Databases → Create database → Standard create를 선택합니다.
3. Engine은 Amazon Aurora → MySQL-compatible, 버전은 8.4.x를 선택합니다. RDS MySQL 단일 인스턴스와 구분합니다.
4. DB cluster identifier는 shgold-mysql입니다. Tags에 Name=shgold-mysql도 추가합니다.
5. Master username과 암호를 정합니다. 이 계정은 스키마 생성에 사용하고 앱 전용 계정은 따로 만듭니다.

6. Instance class는 콘솔에서 8.4와 호환되는 T 계열 medium, 예를 들어 제공되는 경우 db.t4g.medium을 고릅니다. 해당 엔진.리전에서 실제 선택 가능 여부를 확인합니다.
7. Connectivity에서 shgold-vpc, shgold-db-subnets, Public access No, shgold-db-sg를 선택합니다.
8. Initial database name은 shgold를 입력합니다. 생성에 시간이 걸리므로 다음 EFS와 캐시를 준비하며 기다립니다.
9. Available이 되면 Writer endpoint와 포트 3306을 기록합니다.

원문에서 T 계열 medium은 권장 조건입니다. Fargate 태스크에는 t3.medium 같은 EC2 인스턴스 유형이 없으므로 Task CPU-Memory를 설정하는 것이 정상입니다. DB 유형 선택이 안 되면 지원 목록과 정정 기준을 확인하고 자동으로 RDS MySQL로 갈아타지 않습니다.

L5 원본 앱과 데이터베이스의 연결 호환성

Aurora MySQL 8.4는 require_secure_transport=ON이 기본입니다. 지금 ingestion의 pymysql.connect에는 TLS 옵션이 없고 stub 설정에도 TLS 항목이 없습니다. 원본 코드 변경 금지 조건에서 그대로 연결하면 insecure transport 오류가 날 수 있습니다. [S3, S4]

이 실습의 호환 경로는 private DB에 한정하여 사용자 지정 DB cluster parameter group의 require_secure_transport를 OFF로 설정하는 것입니다. 이는 암호화 강제를 완화하는 선택이므로 일반 운영용 기본값으로 권장하는 것은 아닙니다. 대회에서 TLS를 별도 강제하면 앱의 지원 옵션·허용 변경 범위 정정이 필요합니다.

1. RDS → Parameter groups → Create parameter group에서 현재 Aurora MySQL 8.4 엔진에 맞는 family와 DB Cluster Parameter Group 유형을 선택합니다.
2. 이름을 shgold-mysql84-cluster로 정하고 생성합니다.
3. Edit parameters에서 require_secure_transport를 검색해 OFF로 설정하고 저장합니다.
4. Databases → shgold-mysql → Modify → DB cluster parameter group을 새 그룹으로 변경하고 적용합니다.
5. 콘솔의 적용 상태를 확인합니다. 이 매개변수 자체는 동적이지만 새 그룹 연결 상태에서 재부팅 요구가 표시되면 그 상태도 해소합니다.

앱 사용자는 호환성을 위해 mysql_native_password 인증으로 만드는 경로를 뒤 단계에서 보여 줍니다. 공식 8.4 안내에서는 이 플러그인이 활성화되어 있지만 신규 계정의 기본은 caching_sha2_password입니다. 현재 엔진에서 설정이 허용되는지 실제 CREATE USER 결과를 확인합니다. [S3]

L6 EFS 생성과 Access Point

1. EFS → Create file system → Customize에서 이름 shgold-efs, VPC shgold-vpc로 생성합니다.
2. Regional 파일시스템을 선택하고 Network access에 앱이 실행될 두 AZ의 mount target을 둡니다.
3. 각 mount target의 SG를 shgold-efs-sg로 지정합니다. 기본 SG가 불필요하게 남지 않았는지 확인합니다.
4. 파일시스템 생성 후 Access points → Create access point를 누릅니다.
5. Name은 shgold-root, Root directory path는 /shgold, POSIX user UID와 GID는 1000으로 설정합니다.
6. Root directory creation permissions의 owner UID/GID도 1000, permissions는 0755로 지정합니다.
7. File system ID와 Access point ID를 메모합니다.

해설: EFS가 비어 있어도 Access Point가 /shgold를 소유자 권한으로 생성합니다. root 컨테이너의 /mnt/shgold-efs에 이 볼륨이 마운트되면 root가 NFS임을 확인하고 쓸 수 있습니다. 이미지 안에서 mkdir만 해 놓으면 로컬 파일시스템

이므로 거부될 수 있습니다. [S14]

L7 Valkey 캐시 생성

1. ElastiCache → Valkey caches → Create를 누릅니다.
2. Node-based 또는 Design your own cache 경로를 선택합니다. Cluster mode는 Disabled입니다.
3. 이름은 shgold-valkey, 노드 유형은 선택 가능한 T 계열 medium을 예시로 사용합니다.
4. subnet group을 shgold-vpc의 private subnet으로 만들고 SG를 shgold-cache-sg로 설정합니다.
5. 원본 앱의 도움말이 redis://와 host:port를 안내하므로 학습 예제는 TLS 비활성·인증 비활성의 private 전용 캐시로 설명합니다. 외부 접근은 허용하지 않습니다.
6. 노드 1개는 최소 실습 구성입니다. HA를 요구하면 replica와 failover를 별도 고려하되 cluster mode는 Disabled를 유지합니다.
7. Available 후 Primary endpoint와 포트 6379를 기록합니다.

주의: Serverless 또는 cluster mode enabled endpoint를 무작정 넣으면 리다이렉션을 처리하지 못해 캐시 효과가 없을 수 있습니다. TLS·인증을 꺼려면 원본 실행파일의 실제 지원을 먼저 확인해야 하며, 단순히 redis://를 rediss://로 바꾸면 된다고 단정하지 않습니다.

L8 작업용 ARM EC2와 DB 스키마

1. EC2에서 shgold-work를 Amazon Linux 2023 ARM64, t4g.medium으로 만듭니다. private subnet, public IP 없음, shgold-work-sg를 선택합니다.
2. 역할에는 AmazonSSMManagedInstanceCore, 지급파일 버킷 GetObject, 이후 ECR push에 필요한 권한을 부여합니다. 일반 앱 역할과 구분합니다.
3. Session Manager에서 `uname -m`이 aarch64인지 확인합니다.
4. `sudo dnf install -y mariadb105`로 DB 클라이언트를 설치합니다. 실제 AMI 저장소에서 패키지명이 바뀌면 `dnf search mariadb`로 클라이언트 패키지를 확인합니다.
5. `mysql -h DB_WRITER_ENDPOINT -P 3306 -u 관리자이름 -p`로 접속합니다. 암호는 프롬프트에서 입력하고 명령줄에 붙이지 않습니다.
6. 실습파일 module5/schema.sql을 SQL 프롬프트에 붙입니다.

```
CREATE DATABASE IF NOT EXISTS shgold
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE shgold;
CREATE TABLE IF NOT EXISTS readings (
  id CHAR(36) PRIMARY KEY,
  device_id VARCHAR(64),
  metric VARCHAR(64),
  value DOUBLE,
  recorded_at DATETIME,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

7. 아래 REPLACE_DB_PASSWORD를 본인이 만든 앱 암호로 바꿔 실행합니다. SQL 문자열에 작은따옴표가 들어가는 암호는 올바르게 이스케이프해야 하므로 초보 연습에서는 충분히 긴 영숫자·안전한 기호 조합을 사용합니다.

```
CREATE USER 'shgold_app'@'%'
  IDENTIFIED WITH mysql_native_password BY 'REPLACE_DB_PASSWORD';
GRANT SELECT, INSERT ON shgold.readings TO 'shgold_app'@'%';
SHOW CREATE TABLE shgold.readings;
```

완료 확인: 테이블의 PK는 id 하나이며 VARCHAR와 DOUBLE-DATETIME 타입이 원문과 같습니다. 앱 사용자의 %는 DB 사용자 매칭 범위이며 보안 그룹으로 실제 네트워크 접근을 제한합니다. 플러그인 오류가 나면 Aurora 엔진과 지원 설정을 확인하고 약한 인증 우회 코드를 새로 작성하지 않습니다.

L9 비밀값 저장과 실행 역할

1. Secrets Manager → Store a new secret → Other type of secret에서 username=shgold_app, password=앱 암호를 넣습니다.
2. 이름을 shgold/aurora/credentials로 저장하고 ARN을 메모합니다. 같은 리전 eu-central-1인지 확인합니다.
3. ECS task execution role은 ECS Tasks 신뢰로 만들고 AmazonECSTaskExecutionRolePolicy를 연결합니다. 이미지 pull과 로그 전송에 사용합니다.
4. ECS task role은 별도로 만듭니다. stub 역할에는 iam/module5-secret-read.json을 추가합니다.
5. root task role에는 iam/module5-efs.json으로 EFS ClientMount-ClientWrite를 허용합니다. EFS ID와 Access Point ID를 실제 값으로 교체합니다.
6. ingestion용 Lambda 역할에는 기본 로그, AWSLambdaVPCAccessExecutionRole, 동일 secret 읽기 권한을 연결합니다.

고객 관리 KMS 키로 secret을 암호화했다면 소비 역할의 kms:Decrypt와 키 정책도 설정합니다. 기본 Secrets Manager 키를 쓰는 이 실습에서는 불필요한 전체 KMS 권한을 추가하지 않습니다.

역할 구분: execution role은 ECS 에이전트가 ECR과 로그를 다루고, task role은 실행된 앱이 Secrets Manager 같은 AWS API를 호출합니다. stub가 secret AccessDenied인데 execution role만 고치면 해결되지 않을 수 있습니다.

L10 컨테이너 이미지 준비

1. 작업용 ARM EC2에서 원본 root-stub 파일과 실습파일 Dockerfile을 다운로드합니다.
2. 각 폴더에 바이너리, config.ini, Dockerfile을 나란히 둡니다. root 폴더에는 root 바이너리, stub 폴더에는 stub 바이너리입니다.
3. root/config.ini는 port=8080, upstream.port=8081, shared_dir=/mnt/shgold-efs를 유지합니다.
4. stub/config.ini의 database.host는 Writer endpoint, port=3306, dbname=shgold, user=shgold_app으로 바꿉니다.
5. password는 비워 두고 secret_name = shgold/aurora/credentials` 줄의 앞 세미콜론을 제거해 활성화합니다. aws.region=eu-central-1을 확인합니다.
6. 바이너리를 실행해 도움말을 확인합니다. root 실제 서버는 EFS가 없는 작업 EC2에서 정상 시작을 기대하지 않습니다.

```
chmod 755 root/root stub/stub
file root/root stub/stub
./root/root --help
./stub/stub --help
sudo dnf install -y docker
sudo systemctl enable --now docker
```

정상 결과: 두 파일의 형식에 ARM aarch64가 보이고 stub 도움말에 --cache-mode가 나옵니다. Exec format error는 파일 권한이 아니라 CPU 구조 불일치일 수 있습니다.

Dockerfile은 이미 존재하는 실행파일을 포장합니다. 앱 소스를 새로 개발하거나 바이너리를 다시 컴파일하지 않습니다. 비밀번호를 config.ini나 Dockerfile에 넣으면 이미지 계층에 남으므로 secret을 사용합니다.

L11 ECR 저장소와 이미지 push

1. ECR → Private repositories → Create repository에서 shgold-root와 shgold-stub 두 개를 만듭니다.
2. 각 저장소 상세의 View push commands를 엽니다. eu-central-1과 계정 ID가 들어 있는 로그인 명령을 확인합니다.
3. 작업 EC2 역할에 iam/module5-ecr-push.json을 추가합니다. 계정 ID를 교체합니다. GetAuthorizationToken은 Resource *, 이미지 작업은 두 저장소 ARN으로 제한한 예제입니다.
4. 작업 EC2 Session Manager에서 아래처럼 실행합니다. ACCOUNT_ID를 바꿉니다. 현재 디렉터리에 root와 stub 폴더가 있어야 합니다.

```
aws ecr get-login-password --region eu-central-1 | \
  sudo docker login --username AWS --password-stdin \
    ACCOUNT_ID.dkr.ecr.eu-central-1.amazonaws.com
sudo docker build --platform linux/arm64 -t shgold-root:1 ./root
sudo docker build --platform linux/arm64 -t shgold-stub:1 ./stub
sudo docker tag shgold-root:1 \
  ACCOUNT_ID.dkr.ecr.eu-central-1.amazonaws.com/shgold-root:1
sudo docker tag shgold-stub:1 \
  ACCOUNT_ID.dkr.ecr.eu-central-1.amazonaws.com/shgold-stub:1
sudo docker push ACCOUNT_ID.dkr.ecr.eu-central-1.amazonaws.com/shgold-root:1
sudo docker push ACCOUNT_ID.dkr.ecr.eu-central-1.amazonaws.com/shgold-stub:1
```

5. ECR 콘솔에서 tag 1 이미지가 있는지 확인하고 Image URI를 복사합니다.

오류 해결: no basic auth credentials면 같은 sudo docker 환경에 로그인했는지, denied면 EC2 역할 push 권한, exec format error면 ARM 빌드·기본 이미지 구조를 확인합니다. 태그 1을 나중에 덮어썼다면 ECS는 새 태스크로 재배포해야 합니다.

L12 Cloud Map DNS와 ECS 클러스터

1. AWS Cloud Map → Namespaces → Create namespace를 누릅니다.
2. Namespace name은 wsi.local, Instance discovery는 API calls and DNS queries in VPCs 또는 Private DNS를 선택합니다.
3. VPC는 shgold-vpc로 지정하고 생성합니다.
4. ECS → Clusters → Create cluster에서 이름 shgold-cluster를 입력합니다.
5. Fargate를 사용하는 클러스터로 생성하고 Tag Name=shgold-cluster를 추가합니다.

ECS stub service를 생성할 때 Service discovery에서 namespace wsi.local, service name stub, DNS record type A를 지정합니다. 생성 화면에서 Cloud Map 서비스를 직접 만들 수 없다면 Cloud Map namespace 안에 stub 서비스를 먼저 만들고 연결합니다. TTL은 10초처럼 짧게 시작할 수 있습니다.

해설: stub의 태스크 IP는 바뀔 수 있습니다. IP를 root에 하드코딩하지 않고 서비스 등록으로 stub.wsi.local이 현재 IP를 가리키게 합니다. root config에는 호스트 항목이 없으므로 존재하지 않는 upstream.host 설정을 추가하고 작동할 것으로 가정하지 않습니다.

L13 stub Task definition과 서비스

1. ECS → Task definitions → Create new task definition을 누릅니다.
2. Family는 shgold-stub, Launch type Fargate, OS Linux, CPU architecture ARM64를 선택합니다.
3. Task size는 예시로 CPU 0.5 vCPU, Memory 1GB부터 시작합니다. 실행 역할은 ECS execution role, Task role은 secret 읽기 권한을 가진 stub 역할입니다.

4. Container name=stub, Image URI=ECR의 shgold-stub:1, Container port=8081로 설정합니다.
5. Command override에 아래 네 인수를 지정합니다. 화면이 쉼표 분리라면 각각의 값으로 입력하고, JSON 편집에서는 배열을 사용합니다.

```
[ "--config", "/app/config.ini", "--cache-mode", "redis://CACHE_PRIMARY_ENDPOINT:6379" ]
```
6. CloudWatch logs를 켜고 로그 그룹을 생성합니다. 태스크 정의를 저장합니다.
7. 클러스터 → Services → Create에서 Fargate, 방금 정의, Service name=shgold-stub-service, Desired tasks=1을 선택합니다.
8. Network는 private 두 개, public IP Disabled, shgold-stub-sg입니다.
9. Service discovery를 켜고 L12의 namespace wsi.local, service stub, A 레코드로 연결합니다.
10. 배포 후 Running task와 CloudWatch 시작 로그를 확인합니다.

완료 확인: 같은 VPC의 작업 EC2에서 `getent hosts stub.wsi.local`을 실행하면 private IP가 나옵니다. DB 연결 실패는 Aurora SG-사용자-TLS-secret-NAT 순서로 확인합니다. SG가 작업 EC2의 8081 요청을 허용하지 않는 설계라면 작업 EC2에서 직접 curl이 막혀도 DNS 자체는 확인할 수 있습니다.

L14 root Task definition과 EFS 마운트

1. ECS Task definition을 shgold-root로 만듭니다. Fargate, Linux ARM64, CPU 0.5 vCPU, Memory 1GB를 시작값으로 선택합니다.
2. root task role과 ECS execution role을 지정합니다.
3. Storage 또는 Volumes → Add volume에서 Volume name=shgold-shared, Type=EFS를 선택합니다.
4. File system은 L6 EFS, Access point는 shgold-root Access Point, Transit encryption Enabled, IAM authorization Enabled를 설정합니다.
5. Access Point를 사용할 때 Root directory는 / 또는 생략합니다. /shgold를 다시 쓰지 않습니다.
6. Container name=root, Image URI=shgold-root:1, Container port=8080, 로그 활성화를 설정합니다.
7. Container mount points에서 source volume shgold-shared, container path /mnt/shgold-efs, Read only=false로 연결합니다.
8. Task definition을 저장하고 root service를 private 두 subnet, public IP Disabled, shgold-root-sg, Desired tasks=1로 만듭니다.
9. Running을 확인하고 로그에서 공유 디렉터리 검사 실패가 없는지 봅니다.

오류 해결: ResourceInitializationError면 EFS mount target-2049 SG-IAM-Access Point 소유자를 봅니다. 앱 로그에 expected NFS가 있으면 볼륨이 컨테이너 경로에 실제 마운트되었는지 확인합니다. Secret 해결이 느린 stub와 달리 root의 시작 실패는 DB보다 EFS 문제일 수 있습니다.

L15 ingestion Lambda zip과 설정

1. 지급 module5/shgold-ingestion/lambda_function.py와 requirements.txt를 CloudShell에 업로드합니다.
2. 별도의 빈 작업 폴더에서 아래처럼 패키징합니다. module3 index.py와 같은 디렉터리를 재사용하지 않습니다.

```
mkdir -p ~/ingest-build/package
cp ~/lambda_function.py ~/ingest-build/package/
python3 -m pip install --target ~/ingest-build/package \
  -r ~/requirements.txt
cd ~/ingest-build/package
zip -r ../ingestion.zip .
```

3. Lambda → Create function에서 shgold-ingestion, Python 3.12, x86_64를 선택합니다. 이 Python 함수는 ARM 바 이너리와 별개라 x86_64로 실행할 수 있습니다.
4. ingestion 역할을 선택하고 zip을 업로드합니다. Handler는 lambda_function.lambda_handler입니다.
5. Tags에 Name=shgold-ingestion을 넣습니다.
6. 환경변수를 아래 표대로 입력합니다. secret을 사용하므로 DB_PASSWORD는 비워 두거나 생략합니다.

변수	값
DB_HOST	Aurora Writer endpoint
DB_PORT	3306
DB_NAME	shgold
DB_USER	shgold_app
DB_SECRET_NAME	shgold/aurora/credentials
TABLE_NAME	readings
HEALTH_PATH	/healthz

7. Configuration → VPC에서 shgold-vpc, private 두 subnet, shgold-ingest-sg를 연결합니다.
8. 메모리 512MB, Timeout 10초를 시작값으로 둡니다. 실행 로그의 Duration을 보고 조정합니다.

해설: 원본은 모듈 로딩 때 DB_PORT를 int로 변환하고 TABLE_NAME을 검사하므로 /healthz조차 호출 전에 초기화 오류가 날 수 있습니다. 이름만 맞추고 필수 환경변수를 비워 두면 안 됩니다. DB 연결·자격 증명은 warm invocation에서 재사용하는 코드가 이미 포함되어 있습니다.

L16 타깃 그룹 세 개 생성

1. EC2 → Target groups → Create target group에서 shgold-root-tg를 만듭니다. Target type IP addresses, HTTP 8080, VPC shgold-vpc, Health check /healthz입니다.
2. 같은 방식으로 shgold-stub-tg를 IP addresses, HTTP 8081, Health check /healthz로 만듭니다.
3. 세 번째는 Target type Lambda, 이름 shgold-ingest-tg로 만들고 shgold-ingestion 함수를 등록합니다.
4. Lambda 호출 권한을 추가하라는 콘솔 안내를 적용합니다. 함수 Resource-based policy에 elasticloadbalancing.amazonaws.com과 해당 Target group SourceArn이 있어야 합니다.

해설: Fargate awsvpc는 EC2 instance target이 아니라 IP target입니다. 태스크 IP를 수동 고정 등록하지 않고 ECS Service에 타깃 그룹을 연결해 등록·해제를 맡깁니다. Lambda 대상은 DB 포트를 향한 TCP 타깃 그룹이 아닙니다.

L17 단일 ALB와 ECS 연결

1. EC2 → Load balancers → Create → Application Load Balancer를 선택합니다.
2. Name=shgold-alb, Tag Name=shgold-alb, Internet-facing, shgold-vpc, public 두 subnet, shgold-alb-sg로 설정합니다.
3. HTTP 80 listener 기본 동작을 shgold-root-tg로 지정합니다.
4. ALB 생성 후 Listeners and rules → HTTP 80 → Manage rules에서 우선순위 10의 조건 Path=/ingest, 동작 Forward shgold-ingest-tg를 추가합니다.
5. HTTP 8081 listener를 추가하고 shgold-stub-tg로 전달합니다.
6. ECS → root Service → Update → Load balancing에서 기존 shgold-alb, root container 8080, shgold-root-tg를 연결합니다. Health check grace period는 예시로 60초입니다.
7. stub Service도 같은 ALB, stub container 8081, shgold-stub-tg로 연결합니다.
8. 기존 서비스 수정 화면에서 LB 추가가 지원되지 않는 배포 방식이면 새 ECS 서비스를 같은 태스크 정의·네트워크·Cloud Map 설정으로 생성하면서 연결합니다. 이전 서비스의 중복 등록·태스크를 남기지 않도록 상태를 확인합니다.
9. root와 stub 타킷이 Healthy가 된 뒤 ALB DNS를 메모합니다.

Lambda Health check를 커는 경우 원본이 GET /healthz에 응답하므로 해당 경로와 이벤트 형태를 사용합니다. HTTP 80의 /healthz는 root로 가고 HTTP 8081의 /healthz는 stub로 갑니다.

L18 데이터 넣기와 조회

CloudShell에서 아래 명령의 ALB_DNS를 실제 값으로 바꿉니다.

```
curl -i http://ALB_DNS/healthz
curl -i http://ALB_DNS:8081/healthz
curl -i -X POST http://ALB_DNS/ingest \
  -H 'Content-Type: application/json' \
  -d '{"device_id":"category-stuff-1","metric":"temp","value":82.4}'
```

1. POST 응답이 201이며 UUID 형태의 id가 반환되는지 확인합니다.
2. 그 id를 아래 조회 경로에 넣습니다. 문제의 abcd1234는 예시이므로 실제 삽입한 UUID로 확인합니다.

```
curl -i http://ALB_DNS/v1/readings/RETURNED_UUID
```

3. 응답의 device_id-metric-value가 넣은 값과 같은지 확인합니다.
4. DB 작업 세션에서 SELECT id, device_id, metric, value FROM shgold.readings LIMIT 5;로 저장을 확인합니다.

정상 상태: POST와 GET이 같은 데이터를 가리킵니다. root /healthz만 200인 것은 DB 조회 성공 증거가 아닙니다. ingestion /healthz도 원본 구현에서 DB를 조회하지 않으므로 실제 삽입·조회가 필요합니다.

L19 1초 목표를 측정하고 튜닝

먼저 동일 ID를 1회 읽은 뒤 같은 ID를 여러 번 조회합니다. 아래 -w는 HTTP 코드와 전체 소요 시간을 표시합니다.

```
for i in 1 2 3 4 5; do
  curl -s -o /dev/null -w '%{http_code} %{time_total}\n' \
    http://ALB_DNS/v1/readings/RETURNED_UUID
done
```

1. stub Command에 --cache-mode와 올바른 Primary endpoint가 실제 반영되었는지 확인합니다. 태스크 정의 새 revision만 만들고 서비스가 이전 revision을 쓰는 실수를 확인합니다.
2. cache cluster mode Disabled, 6379 SG, DNS, endpoint를 확인합니다.
3. root-stub access log의 upstream_ms-duration_ms 같은 시간 단서를 실제 로그에 존재하는 범위에서 비교합니다.
4. 같은 ID의 첫 조회와 반복 조회를 따로 기록합니다. 도움말에 따르면 이미 캐시된 조회는 DB를 건너뛰지만 처음 보는 ID는 DB 접근이 필요합니다.
5. ingestion 첫 호출과 warm 호출을 나눠 측정합니다. 필요하다면 버전을 Publish하고 alias를 만든 뒤 Provisioned concurrency를 alias에 적용합니다. 이 경우 ALB Lambda 타킷도 그 alias ARN으로 등록해야 효과가 있습니다.
6. DB CPU·연결 수·쿼리 시간, ECS CPU·메모리를 확인합니다. 원문 PK id 조회에서 근거 없이 추가 인덱스만 만드는 것은 지연 해결의 증거가 아닙니다.

중요: cache hit만 빠르다고 전체 목표를 달성했다고 보고하지 않습니다. 새 ID 조회가 계속 1초를 넘으면 cold read 한계가 남은 것입니다. DB 대기·앱 내부 지연을 로그로 구분하고, 변경 허용 범위 내 옵션을 추가 확인해야 합니다. 소스 없는 바이너리의 지연을 무조건 DB 크기만 늘려 해결된다고 단정하지 않습니다.

측정은 인터넷 클라이언트 기준과 동일 리전 기준이 다를 수 있습니다. 성공률, 첫 호출, 반복 호출, 표본 수, 호출 위치를 같이 기록합니다. 5회 예제는 기능 점검용이며 대회의 실제 부하 조건을 대신하지 않습니다.

L20 장애별 확인 순서

증상	확인 순서
ECS가 바로 종료됨	ARM64 → 실행 권한 → config 경로 → 시작 로그
root가 NFS 오류로 종료	EFS volume → mount path → Access Point → 2049 SG
root health 성공, 조회 실패	stub.wsi.local DNS → 8081 SG → stub DB 연결
stub secret 처리 시간 초과	task role → secret 리전·이름 → NAT 또는 VPC endpoint
ingestion 초기화 오류	DB_PORT 3306 → TABLE_NAME readings → zip 의존성
DB insecure transport 오류	Aurora 8.4 TLS 기본값과 원본 앱 호환 설정
ALB 502	Lambda 로그·응답 형태 또는 컨테이너 연결 오류
반복 조회도 느림	실제 실행 Command의 cache-mode → cluster mode → 6379
ALB health만 정상	반드시 POST 삽입 후 반환 ID로 GET 조회

제출 전 ECR push 작업을 끝내도 ECS 실행 역할의 pull 권한은 유지합니다. 서비스가 재시작했을 때 이미지를 다시 가져와야 합니다. 작업 EC2의 과도한 임시 권한은 역할을 검토해 제거하고, 채점용 Bastion 접근 경로는 유지합니다.

7 제출 전 채점 항목별 확인

이 표의 배점은 첨부 채점표를 옮긴 것입니다. 체크했다고 자동으로 점수가 보장되는 것은 아닙니다. 실제 채점은 제공된 채점 환경과 정정 기준에 따릅니다.

Workflow 7점

항목	점수	확인할 증거
S3 Bucket와 Folder Structure	1	정확한 이름·리전·prefix, 제출 시 완전히 빈 버킷
DynamoDB Table와 Key Schema	1	테이블 이름, studentId String + examDate String, 추가 키 없음
Lambda Function와 Runtime과 Env	1	함수 이름, Python 3.12, S3_BUCKET·DDB_TABLE
Step Functions State Machine	1	정확한 이름, Standard, 지정 흐름·Retry·분기
Workflow Result Normal	1.5	새 업로드 자동 실행, 정상 5행·평균·등급·processed 파일
Workflow Result Error	1.5	오류 행 JSON 4개, 오류 사유, 실패 경로 처리

최종 확인은 버킷과 테이블이 빈 상태인지 다시 보는 것입니다. 검증용 데이터가 남으면 원문의 1-1·1-5·1-6 채점에 불이익이 생깁니다. 인프라는 삭제하지 않습니다.

Real time data analytics 7점

항목	점수	확인할 증거
EC2 Instance	0.5	지정 이름, t3.small, private-a, SSM 접속
ALB Resources	1	이름, HTTP 80, TG 5000, /health, healthy
Kinesis Stream	1	wsc2026-order-stream, On-demand
Kinesis Data	1	주문 JSON 필드와 실제 새 레코드
Flink Application	1	Studio Notebook, 이름, 버전 정정 여부, SQL 두 개
Application Health	1	ALB /health 200과 주문 API 201
Systemd Service	1.5	app active-enabled, 재부팅 후 자동 실행

Flink 지원 버전 충돌은 해결 여부를 기록합니다. 일반 Flink 앱이 Running인 화면만으로 Studio 요구를 충족했다고 표시하지 않습니다.

MSK 7점

항목	점수	확인할 증거
Resources	0.5	도쿄 VPC·EC2·DynamoDB·S3 등 요구 리소스
Lambda Functions	1.5	두 함수 이름, Python 3.14, 핸들러·변수·권한
MSK Cluster Configuration	1.5	private, IAM only, Kafka 3.6.0 또는 정정 버전, kafka.t3.small
MSK Trigger Mapping	1.5	raw-alert 정확한 연결과 Enabled
Data Processing Result	1	NORMAL DB, ALERT Kafka→S3·SNS, 타입·KST
Producer Running	1	원본 producer 지속 실행과 재부팅 복구

센서 producer는 지속 실행 조건이므로 임시 부하 테스트 중지와 혼동하지 않습니다. 이메일 확인을 못 했을 때는 Lambda Publish 오류 여부와 SNS 지표를 함께 확인합니다.

CDN Service Setup 3점

항목	점수	확인할 증거
CloudFront OAI	1	OAI와 KMS 충돌에 대한 공식 정정, 선택한 접근 방식
Cache Invalidation	1	S3 덮어쓰기 → 자동 무효화 → 3분 내 브라우저 반영 측정
API Security	1	CloudFront 경유 성공, 비밀번호 없는 직접 API 요청 거부

API Security의 구체적인 세부 검증 방법은 사진에 없습니다. 공유 비밀 헤더 방식은 이 해설서의 구현안입니다. 채점 기가 다른 인증 방식이나 조건을 명시하면 그 기준을 적용합니다.

Legacy System Operation 6점

항목	점수	확인할 증거
Service Provisioning	1	이름·Name tag, Fargate ARM64, Aurora 8.4, 하나의 ALB
Data Ingestion	1	POST /ingest 201, UUID, DB 실제 저장
Success Rate	1	넣은 UUID 조회 성공, 반복 요청 오류 비율
Performance Tuning	1.5	캐시 전후, cold-warm 구분, 1초 목표 측정
Serverless Computing	1.5	root-stub Fargate, ingestion Lambda

캐시 hit만 성공한 측정과 새 ID까지 성공한 측정을 구분합니다. 최초 조회가 목표를 넘으면 미해결 성능 조건으로 남겨야 합니다.

공통 제출 점검 순서

1. 각 과제의 리전과 요구 이름을 사진과 한 줄씩 대조합니다. 특히 wsc와 wsk를 확인합니다.
2. 채점용 Bastion을 생성하고 관리 경로를 점검합니다. 사진에는 구체적인 Bastion 리전·단일 서버 여부가 없으므로 운영 지시가 있으면 그대로 적용합니다.
3. 이 해설서의 분리 VPC 구성에서는 각 VPC에 SSM으로 접근 가능한 관리 EC2를 두는 방식을 사용할 수 있습니다. Analytics와 MSK 서버를 관리 지점으로 활용할 경우 채점자가 허용하는지 확인합니다.
4. 한 Bastion에서 모든 리전의 private 리소스에 직접 TCP 접근해야 한다면 별도의 inter-region VPC peering 또는 Transit Gateway와 양방향 경로·SG가 필요합니다. 단순히 IAM AdministratorAccess를 붙여도 네트워크가 연결되지 않습니다. 채점 접근 방식이 미정인 상태에서 이것이 충족되었다고 가정하지 않습니다.
5. 서버의 SSM 접속, DB·Kafka·EFS·캐시 관리 접근, AWS API 조회 권한을 채점 방식에 맞게 확인합니다.
6. Name tag, 보안 그룹, 실행 역할, 로그 접근 권한을 점검합니다.
7. 일회성 테스트·부하 루프를 종료합니다. app-producer·ECS 서비스·Lambda 트리거는 필요한 상태로 유지합니다.
8. Workflow S3와 DynamoDB만 요구대로 빈 상태로 만들고 마지막으로 다시 조회합니다.
9. 계정 비밀번호나 secret을 포함하지 않은 설정 증거와 정상 결과를 저장합니다.

8 초보자를 위한 오류 해결 사전

화면에 리소스가 보이지 않을 때

오른쪽 위 리전을 먼저 봅니다. IAM 역할은 전역이지만 Lambda-EC2-DynamoDB는 리전별입니다. 이름 검색에 공백이 섞이거나 필터가 남아 있으면 지우고 다시 검색합니다. 이름이 비슷한 다른 과제 리소스를 삭제하지 않습니다.

AccessDenied와 연결 시간 초과를 구분

AccessDenied-not authorized는 보통 IAM 또는 resource policy 문제입니다. 오류의 Action, Resource, 사용 역할을 기록하고 그 세 가지를 대조합니다. timeout:connection refused-name resolution failure는 네트워크, 포트, 프로세스, DNS 문제일 가능성이 큼니다. 권한을 계속 늘리는 대신 오류 종류에 맞게 확인합니다.

코드를 붙였는데 이전 결과가 나올 때

Lambda는 Deploy를 눌렀는지, ECS는 새 Task definition revision을 서비스에 적용했는지, systemd는 daemon-reload와 restart를 했는지 확인합니다. CloudFront 설정은 배포 완료까지 전파 시간이 필요합니다. 각 서비스의 적용 시점을 구분합니다.

Handler가 무엇인지 모를 때

Python의 파일명에서 .py를 뺀 부분과 함수명을 점으로 잇습니다. index.py에 def handler가 있으면 index.handler입니다. lambda_function.py의 def lambda_handler면 lambda_function.lambda_handler입니다. 코드에 함수가 있어도 zip 최상위 파일 경로가 다르면 실패합니다.

systemd가 enabled인데 앱이 실행되지 않을 때

enabled는 부팅 시 시작하도록 등록되었다는 뜻이고 active는 현재 실행 중이라는 뜻입니다. 둘 다 확인합니다. 파일을 수정한 뒤 `sudo systemctl daemon-reload`와 `sudo systemctl restart app`을 실행합니다. journalctl의 마지막 오류를 먼저 읽습니다.

ALB가 Healthy인데 실제 요청이 실패할 때

헬스체크는 특정 경로가 응답하는지만 봅니다. 이 문제의 /health와 /healthz는 DB-Kinesis까지 깊게 검사하지 않을 수 있습니다. 항상 실제 주문 생성, 데이터 삽입, 데이터 조회 요청을 추가로 수행합니다.

파일은 있는데 Permission denied가 나올 때

바이너리는 `chmod 755`로 실행 권한을 주고 부모 디렉터리 접근 권한도 봅니다. EFS에는 네트워크 2049, IAM, POSIX 소유자 권한의 세 층이 있습니다. 모든 파일에 `chmod 777`을 적용하기 전에 어느 층에서 거부되는지 확인합니다.

JSON과 셸 명령 복사 규칙

JSON에는 주석이나 마지막 쉼표를 넣지 않습니다. 문자열에는 일반 큰따옴표를 사용합니다. 셸의 줄 끝 역슬래시는 다음 줄과 이어진다는 뜻이며 뒤에 공백이 붙지 않게 합니다. Python은 들여쓰기를 유지합니다. 본문에서 생긴 줄바꿈 때문에 오류가 나면 동봉 원본 코드 파일을 사용합니다.

실습을 마친 뒤 비용 정리

대회 제출 전에는 채점 리소스를 삭제하지 않습니다. 개인 연습 종료 후에만 본인이 만든 리소스를 확인하여 삭제합니다. MSK, Aurora, ElastiCache, NAT, ALB, 실행 중 Flink, EC2, Fargate는 켜 둔 상태에서 비용이 계속 발생할 수 있습니다. EC2를 중지하는 것만으로 다른 서비스가 정지되지는 않습니다.

정리할 때는 서비스·태스크를 먼저 중지하고 종속 리소스를 확인합니다. RDS snapshot·EBS·EIP·S3 객체·ECR 이미지·CloudWatch 로그도 남을 수 있습니다. 이 해설서는 계정의 기존 자원을 자동 삭제하는 스크립트를 제공하지 않습니다.

9 참고 자료와 검증 범위

원문과 지급파일의 대응

원문 범위	대조한 지급파일	해설서 반영
2~3쪽 Workflow	module1/lambda.md, workflow.md, lambda-function.py, test.csv	TODO 완성, 5개 정상·4개 오류, 분기와 이동
3~5쪽 Analytics	module2/Application.md, app.py, requirements.txt	Python 3.12, 환경변수, API, UTC, systemd
5~6쪽 MSK	module3/Application.md, lambda.md, app	임계치, SNS, S3 경로, x86_64
7~8쪽 CDN	module4/image.png	이미지 업로드와 자동 무효화
9~10쪽 Legacy	module5의 config.ini, 바이너리, ingestion Python	ARM64, EFS, DNS, cache-mode, DB 환경변수
채점표 2장	사진의 배점 표	총 30점 체크리스트

문서에 인용된 과제 지시는 학습용 요구사항입니다. 이 문서를 만들면서 실제 AWS 자원 생성, 외부 알림 발송, 계정 데이터 삭제는 수행하지 않았습니다. 실행 예제는 사용자가 자신의 실습 계정에서 단계별로 적용하도록 작성했습니다.

AWS와 프로젝트 공식 문서

[S1] CloudFront의 S3 접근 제한과 OAI·OAC 기능 차이 <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html>

[S2] Managed Flink 1.19의 Studio 지원 제한 <https://docs.aws.amazon.com/managed-flink/latest/java/flink-1-19.html>

[S3] Aurora MySQL 8.4 엔진과 인증 기본값 <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/AuroraMySQLMySQL84.html>

[S4] Aurora MySQL TLS와 require_secure_transport <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/AuroraMySQLSecurity.html>

[S5] Managed Flink 1.15와 Studio 구성 <https://docs.aws.amazon.com/managed-flink/latest/java/flink-1-15-2.html>

[S6] Studio Kinesis 테이블과 SQL 예제 <https://docs.aws.amazon.com/managed-flink/latest/java/how-zeppelin-sql-examples.html>

[S7] Amazon MSK 버전 지원 정책 <https://docs.aws.amazon.com/msk/latest/developerguide/version-support.html>

[S8] Apache Kafka 3.6.0 공식 배포 파일 <https://archive.apache.org/dist/kafka/3.6.0/>

[S9] AWS MSK IAM 인증 Java 라이브러리 릴리스 <https://github.com/aws/aws-msk-iam-auth/releases>

- [S10] Lambda MSK 이벤트 소스 권한 <https://docs.aws.amazon.com/lambda/latest/dg/with-msk-permissions.html>
- [S11] AWS Lambda 지원 런타임 <https://docs.aws.amazon.com/lambda/latest/dg/lambda-runtimes.html>
- [S12] Lambda MSK 네트워크와 poller 연결 <https://docs.aws.amazon.com/lambda/latest/dg/with-msk-cluster-network.html>
- [S13] CloudFront 관리형 Origin Request Policy <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/using-managed-origin-request-policies.html>
- [S14] ECS Task의 EFS 볼륨과 마운트 설정 <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/specify-efs-config.html>
- [S15] ECS 서비스 검색과 Cloud Map <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/service-discovery.html>
- [S16] AWS Python MSK IAM signer 사용법 <https://github.com/aws/aws-msk-iam-sasl-signer-python>
- [S17] CloudFront 무효화 동작 <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Invalidation.html>

현장 확인이 필요한 부분

메뉴 명칭, 계정 할당량, 리전별 인스턴스·엔진 선택지, 서비스 버전 지원은 실제 콘솔에서 확인합니다. CDN OAI·KMS와 Flink Studio 버전의 원문 충돌은 기술 설명만으로 채점 기준을 확정할 수 없습니다. Legacy 성능은 실제 데이터·부하와 최초 조회를 포함해 측정해야 합니다.

학습 완료의 기준은 화면에 자원이 존재하는 것이 아니라, 새 입력이 의도한 경로를 지나 올바른 결과를 만드는 것입니다. 각 장의 정상 결과와 오류 확인 순서를 이용해 자신의 계정에서 재현하세요.